Imperial College
London

COURSEWORK

IMPERIAL COLLEGE LONDON

DEPARTMENT OF ELECTRICAL AND ELECTRONIC ENGINEERING

Advanced Signal Processing

Author:
Harry Davies (CID: 00966288)

Professor:
Danilo P. Mandic

Very tidy, well written and nicely formated report. The student showed deep understanding of the material. Easy to follow all explanations. All conclusions are supported with appropriate figures and mathematical derivations.

Date: March 23, 2017

Contents

1 Random signals and stochastic processes 3
- 1.1 Statistical estimation . . . 3
- 1.2 Stochastic processes . . . 6
- 1.3 Estimation of probability distributions . . . 9

2 Linear stochastic modelling 10
- 2.1 ACF of uncorrelated sequences . . . 10
- 2.2 ACF of correlated sequences . . . 11
- 2.3 Cross-correlation function . . . 12
- 2.4 Autoregressive modelling . . . 13
- 2.5 Real world signals: ECG from iAmp experiment . . . 16

3 Spectral estimation and modelling 18
- 3.1 Averaged periodogram estimates . . . 18
- 3.2 Spectrum of autoregressive processes . . . 19
- 3.3 Spectrogram for time-frequency analysis: dial tone pad . . . 22
- 3.4 Real World Signals: Respiratory sinus arrhythmia from RR-Intervals . . . 25

4 Optimal filtering - fixed and adaptive 26
- 4.1 Wiener filter . . . 26
- 4.2 The least mean square (LMS) algorithm . . . 28
- 4.3 Gear shifting . . . 30
- 4.4 Identification of AR processes . . . 31
- 4.5 Speech recognition . . . 32
- 4.6 Dealing with computational complexity: sign algorithms . . . 35

5 A Real World Case Study: Vinyl Denoising 36

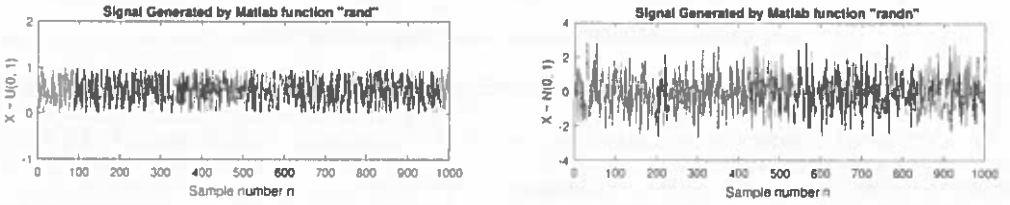
Good table of contents, nice and tidy

1 Random signals and stochastic processes

In this section we study the statistical properties of stochastic signals. To design systems that operate in the real world we must have a strong understanding of the statistical properties that underlie real-world signals, and these always have stochastic elements.

1.1 Statistical estimation

We now analyze the statistical properties of a vector containing 1000 random samples: $x = [x[1], x[2], ..., x[1000]]^T$. Firstly this is generated with uniform random variables $X \sim U(0,1)$, and secondly with Gaussian random variables of mean 0 and standard deviation 1, $X \sim N(0,1)$.



(a) Uniformly random samples

(b) Gaussian random samples

Figure 1.1.1: Signals generated containing 1000 random samples.

The probability density function for a uniform random variable between 0 and 1 is given by:

$$f_U(x) = \begin{cases} 0 & \text{if } x < 0, x > 1 \\ 1 & \text{if } 0 \leq x \leq 1 \end{cases} \tag{1}$$

The probability density function for a Gaussian random variable is given by:

$$f_N(x) = \frac{1}{\sqrt{2\pi}\sigma} e^{-\frac{(x-\mu)^2}{2\sigma^2}}, \quad -\infty < x < \infty \tag{2}$$

Question 1 and 2

Theoretical mean and standard deviation:

The theoretical mean of $U \sim X(0,1)$ is given by:

$$E\{X\} = \int_0^1 x \cdot f_U(x)dx = [\tfrac{1}{0}\frac{x^2}{2}] = 0.5$$

The theoretical mean of $N \sim X(0,1)$ is given by:

$$E\{X\} = \int_{-\infty}^{\infty} x \cdot f_N(x)dx = \mu = 0$$

The theoretical standard deviation of $U \sim X(0,1)$ is given by:

$$\sqrt{E\{X^2\} - E\{X\}^2} = \sqrt{(\int_0^1 x^2 \cdot f_U(x)dx) - (0.5)^2}$$

$$= \sqrt{[\tfrac{1}{0}\frac{x^3}{3}] - 0.25} = \sqrt{\frac{1}{3} - \frac{1}{4}}$$

$$\therefore = 0.2887$$

Equations are nice and tidy

The theoretical standard deviation of $N \sim X(0,1)$ is given by:

$$\sqrt{E\{X^2\} - E\{X\}^2} = \sqrt{(\int_{-\infty}^{\infty} x^2 \cdot f_N(x)dx) - \mu^2} = \sigma = 1$$

The theoretical values, and estimations from 1000 random samples, of mean and variance are given for both the uniform random variable and Gaussian random variable in **table 1.1.1.**

Accuracy of statistical estimators:

The measure of accuracy used for the sample mean is the central limit theorem, through calculating the z score: $z = \frac{\hat{m} - m}{\frac{\sigma}{\sqrt{N}}}$

The measure of accuracy used for the sample standard deviation is the percentage error:
$e = |((\sigma - \hat{\sigma})/\sigma).100|$ From **table 1.1.2** we can conclude that in this case the sample mean of the uniform distribution

Random Variable	Mean		Standard Deviation	
	Theoretical	Estimation	Theoretical	Estimation
	$m = E[X]$	$\hat{m} = \frac{1}{N}\sum_{n=1}^{N} x[n]$	$\sigma = \sqrt{E[X^2] - [E[X]]^2}$	$\hat{\sigma} = \sqrt{\frac{1}{N-1}\sum_{n=1}^{N}(x[n] - \hat{m})^2}$
$X \sim U(0;1)$	0.5	0.5074	0.2887	0.2906
$X \sim N(0,1)$	0	-0.0632	1	0.9990

Table 1.1.1: Summary of mean and standard deviation values. (to 4.s.f)

Correct theoretical and sample mean and std

Random Variable	Accuracy	
	$\hat{m}$ (z score)	$\hat{\sigma}$ (percentage error)
$X \sim U(0;1)$	0.8101	0.6581%
$X \sim N(0,1)$	-1.999	0.1%

Good use of the table

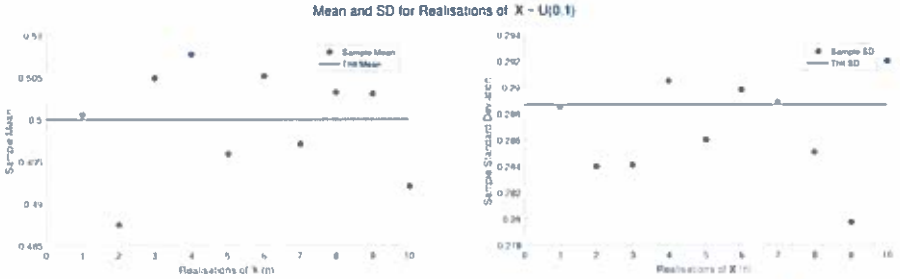
Table 1.1.2: Summary of estimator accuracy. (to 4.s.f)

is more accurate than the sample mean of the Gaussian distribution in terms of their individual theoretical standard deviations (0.81 z score against −1.9 z score). However, the sample standard deviation of the Gaussian distribution is more accurate than the sample standard deviation of the uniform distribution (0.1% error against 0.6581% error). The standard deviation of the uniform distribution is less, which can explain why the accuracy of the sample mean is greater, and the relative percentage error of the standard deviation is worse.
Our estimator for variance includes a factor of $\frac{1}{N-1}$ known as Bessel's correction, which removes the bias in the variance estimator, and some of the bias in the estimation of standard deviation. However, Bessel's correction can also lead to an increase in mean squared error.

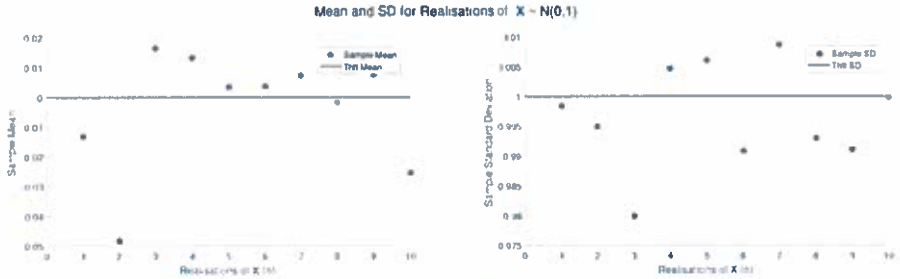
Correct explanations of sample mean and std as estimators

Question 3

Legend and labels could've been bigger



(a) Estimators from uniformly random samples compared with theoretical values



(b) Estimators from Gaussian random samples compared with theoretical values

Figure 1.1.2: Estimators from ensembles of 10 groups of 1000 samples, per random variable type.

Bias of sample mean is calculated with $B = E[X] - \hat{m}$, and similarly for the standard deviation as $B = \sqrt{E[X^2] - [E[X]]^2} - \hat{\sigma}$. Observation of **figure 1.1.2** and **table 1.1.3** confirms that values of mean and standard deviation cluster around their theoretical values. Average estimator bias is less in the case of the uniform distribution, but this is once again due to the standard deviation being smaller (0.2887 for the uniform distribution compared with 1 for the Gaussian distribution).

Good explanation of bias

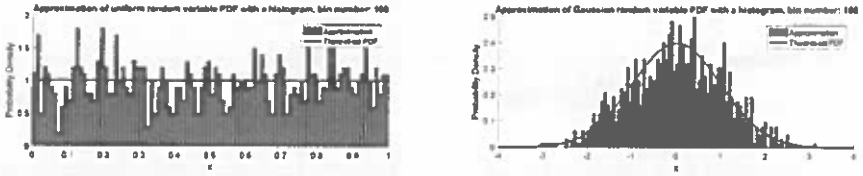
$X \sim U(0,1)$		$X \sim N(0,1)$	
$\hat{m}$ bias	$\hat{\sigma}$ bias	$\hat{m}$ bias	$\hat{\sigma}$ bias
0.0003	0.0018	0.0037	0.0032

Table 1.1.3: Mean average of estimator bias all 10 trials of 1000 random samples. (to 4.s.f)

Question 4

A P.D.F (probability density function) allows us to obtain a complete statistical picture of a variable. The integral across an interval of a probability density function gives us the probability that the value of a random variable will fall within that interval. This leads to the rule that the integral over the whole PDF should always be equal to 1.

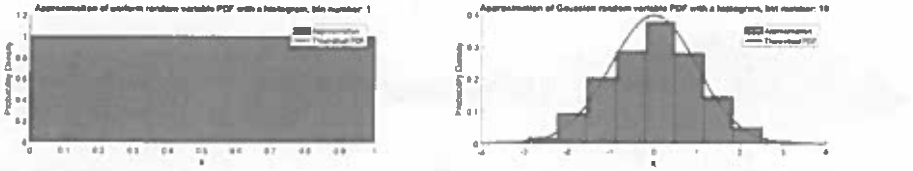
Correct plots of pdf and good choice of bin sizes.



(a) Uniformly random samples

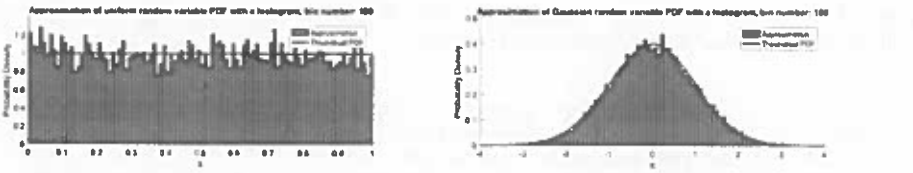
(b) Gaussian random samples

Figure 1.1.3: PDF approximation using 1000 samples and 100 bins.



(a) Uniformly random samples (1000 samples, 1 bin)

(b) Gaussian random samples (1000 samples, 10 bins)



(c) Uniformly random samples (10000 samples, 100 bins)

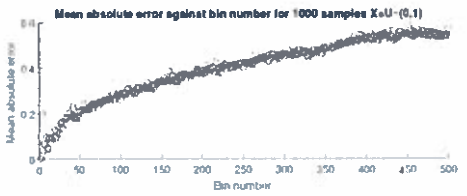
(d) Gaussian random samples (10000 samples, 100 bins)

Figure 1.1.4: Variance in the number of histogram bins (a,b), and variance in the number of samples (c,d).

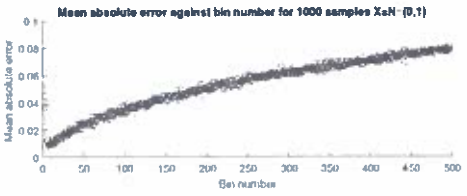
Firstly, we can see from comparison of **figure 1.1.4 a, b and figure 1.1.3** that a large increase in the number of bins from 1 or 10, to 100, leads to a less accurate approximation. This is because each bin's height is averaged over less samples, and therefore the bin height is subject to more variation. However, if the PDF isn't uniform as in the case of the Gaussian distribution, then there must be a limit to how low the optimal bin number will be due to the nature of the bins being square and unable to fully approximate curves. **Figure 1.1.5 a and b** confirm this by showing an overall increase in error as bin number increases, but a sharp decrease in error very early on in the case of the Gaussian random variable.
Secondly, we can see from comparison of **figure 1.1.4 c, d and figure 1.1.3** that an increase in number of samples with bin number constant leads to a better approximation from the histogram. This is due to the same reason as previously. There are more samples per bin, and therefore the average probability density over them is closer to the real value. **Figure 1.1.5 b and c** confirm this by showing a decrease in error as samples increase, but towards a finite value in the case of the Gaussian random variable as the error becomes limited by the number of bins due to its curvature. In this case for a very large sample number, an increase in bin number is actually needed to further improve accuracy.

Good explanation of affect of increasing number of bins/samples

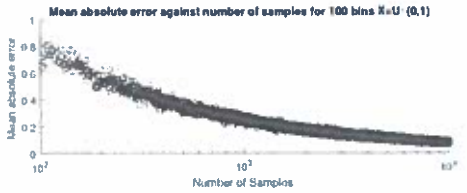
Question 5 is contained within questions 1,2,3 and 4 as the Gaussian analysis.



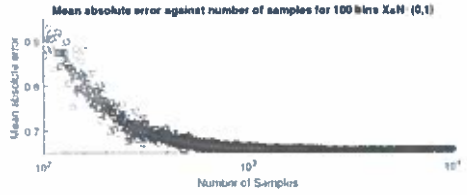
(a) $U(0,1)$ mean absolute error against bin number



(b) $N(0,1)$ mean absolute error against bin number



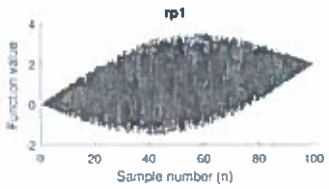
(c) $U(0,1)$ mean absolute error against sample size



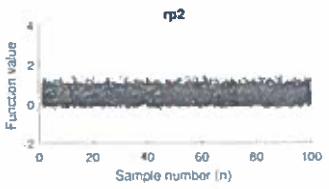
(d) $N(0,1)$ mean absolute error against sample size

Figure 1.1.5: Mean absolute error in histogram approximation with varying bin number and sample size.

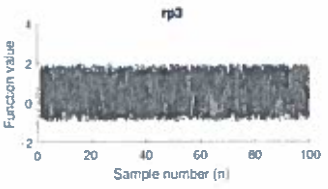
1.2 Stochastic processes



(a) 100 generations of rp1



(b) 100 generations of rp2



(c) 100 generations of rp3

Figure 1.2.1: Realisations of given functions rp1, rp2, and rp3 (each of the many colours corresponds to one of the 100 individual runs of each given function) .

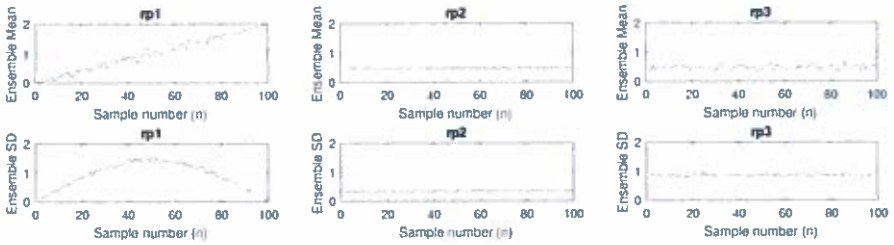
In this subsection we calculate time averages and ensemble averages for 3 different stochastic processes defined as **rp1**, **rp2**, and **rp3**. An ensemble average is the average signal over a number of trials. It is calculated by finding the mean value of each individual time point over all the trials (M), given by:

$$m_x(n) = \frac{1}{M}\sum_{i=1}^{M} x_i(n)$$

In contrast, a time average is just the average value over a single realization of the signal. For an individual wave form a mean is calculated using every time point.

$$m_x(n) = \frac{1}{N}\sum_{n=0}^{N-1} x(n)$$

Question 1



Correct plots of mean and std for all processes

Figure 1.2.2: Ensemble averages of each function

correct definition of a stationary process

A strict sense stationary process is one of which the all the moments of the data are invariant to time shifts. This is too strict for our application, and therefore we refer to stationarity in the sense of wide sense stationary. A processes is wide sense stationary if it's mean is constant, and its autocorrelation $r_{xx}(k,l)$ depends only on the difference $k-l$.[1] We can clearly deduce from **figure 1.2.2** that rp1 is non stationary, as its mean and standard deviation change over time. However, rp2 and rp3 have relatively constant means and standard deviations and are therefore stationary. This is further shown by visually examining the partial correlations in **figure 1.2.3**. Rp1 has an ACF that which gradually decays and stays far beyond the significance range, which indicates non-stationarity.

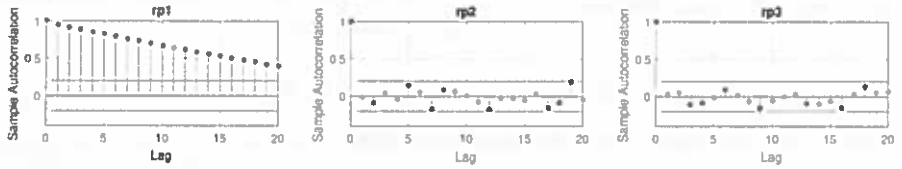


Figure 1.2.3: Partial correlation of ensemble averaged signals (red), the blue line corresponds to a statistically significant bound positioned at 0.2

Question 2

Correct computation of mean and std,
Correct definition of ergodicity.

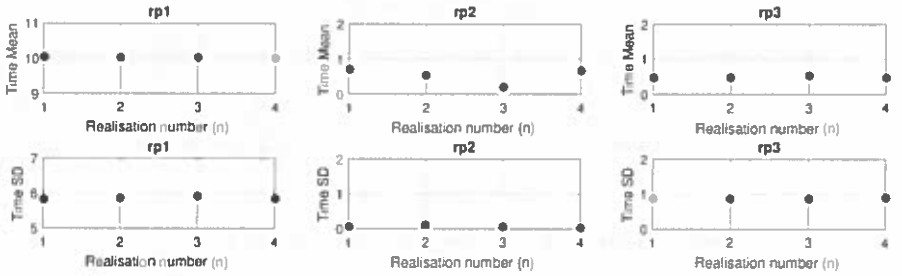


Figure 1.2.4: 4 independent time averages of each function (1000 samples)

An ergodic process is a process of which you can infer its statistical properties from one sufficiently long sequence. **figure 1.2.4** shows us that mean and standard deviation of rp2 vary heavily between each realisation, and therefore one sequence is not enough to infer it's statistical properties; rp2 is not ergodic. This is further shown in **table 1.2.1**, where the estimator variance of rp2 estimators is orders of magnitude higher than in rp1 and rp3. Rp3 has very low variance in its estimators, and we know from **figure 1.2.2 and 1.2.3** that it also has stationary statistical properties; rp3 is ergodic.

Function	$\hat{m}$ variance	$\hat{\sigma}$ variance
rp1	0.0014	0.0003
rp2	0.0717	0.0072
rp3	0.0008	0.0002

Table 1.2.1: Varience of estimators over 20 realizations of 1000 samples. (to 4.d.p)

Rp1 has a very low variance in its estimators with each realisation, however it also has non stationary statistical properties. Changing the sample number, no matter how long the sequence, will change the statistical properties. Therefore, one sufficiently long sequence is not enough; rp3 is not ergodic. This is confirmed in **figure 1.2.5** which shows a change in time averaged statistics as the sample length increases.

Inaccurate identification of which process is ergodic

[1] Statistical Digital Signal Processing and Modeling - M.H.Hayes, 1996, Page 82

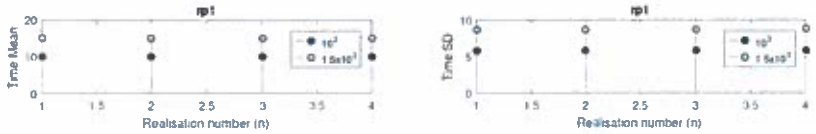


Figure 1.2.5: Comparison of time averaged estimators for rp1 (1000 samples and 1500 samples).

Question 3

Mathematical description of each function:

Correct mathematical representation of all processes

Rp1:

$$v[n] = \left[U[n](-0.5, 0.5) \cdot bsin(\frac{n\pi}{N})\right] + an$$

where $b = 5$, and $a = 0.02$, and $U[n](-0.5, 0.5)$ changes with each new n.

The expected value of $U[n](-0.5, 0.5)$ is zero, and therefore we are left with:

$$\mu = 0.02n$$

When a random variable is scaled, the variance is multiplied by the square of that scale factor. In this case:

$$\sigma^2 = \frac{25}{12} \cdot sin^2(\frac{n\pi}{N}), \quad \sigma = \frac{5\sqrt{3}}{6} \cdot sin(\frac{n\pi}{N})$$

In summary, the theoretical standard deviation is $\frac{5\sqrt{3}}{6} \cdot sin(\frac{n\pi}{N})$, and the theoretical mean is $0.02n$.

Correct analytical estimation of mean and std for all processes

Rp2:

$$v[n] = \left[U[n](-0.5, 0.5) \cdot U(0,1)\right] + U(0,1)$$

where $U[n](-0.5, 0.5)$ changes with each new n, but U(0,1) is constant from the initial value onwards.

$$E\{v\} = E\{U(-0.5, 0.5)\} \cdot E\{U(0,1)\} + E\{U(0,1)\} = 0 + \frac{1}{2} = 0.5 = \mu$$

$$E\{v^2\} = E\{U^2(-0.5, 0.5)\} \cdot E\{U^2(0,1)\} + E\{U^2(0,1)\} = (\frac{1}{12} \cdot \frac{1}{3}) + \frac{1}{3} = \frac{13}{36}$$

$$\sigma^2 = \frac{13}{36} - \frac{1}{2}^2 = \frac{1}{9}, \quad \therefore \sigma = \frac{1}{3}$$

In summary the theoretical standard deviation is $\frac{1}{3}$, and the theoretical mean is 0.5.

Rp3:

$$v[n] = \left[U[n](-0.5, 0.5) \cdot m\right] + a$$

where $m = 3$, $a = 0.5$, and $U[n](-0.5, 0.5)$ changes with each new n.

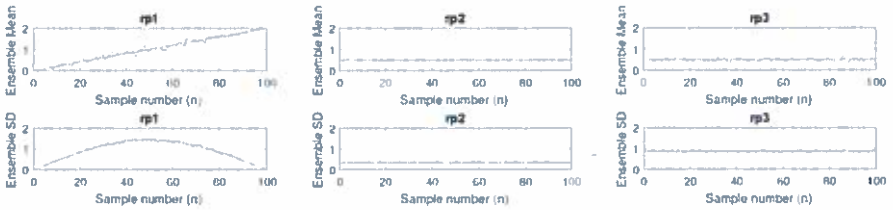
The expected value of $U[n](-0.5, 0.5)$ is zero, and therefore we are left with:

$$\mu = a = 0.5$$

Once again the random variable is scaled, in this case by 3, giving:

$$\sigma^2 = 9 \cdot \frac{1}{12} = \frac{3}{4}, \sigma = \frac{\sqrt{3}}{2}$$

In summary, the theoretical standard deviation is $\frac{\sqrt{3}}{2}$, and the theoretical mean is 0.5.



Figures could've been bigger

Figure 1.2.6: Ensemble averages (blue) of each function compared with theoretical values (red).

1.3 Estimation of probability distributions

This subsection involves the development of a MATLAB function `pdf` which uses the inbuilt function `hist` to approximate probability density distributions. This is performed by normalizing the histogram area to 1, as this ensures that the PDF has an area of 1.

No code for pdf is given in the report

Question 1

Gaussian Approximation

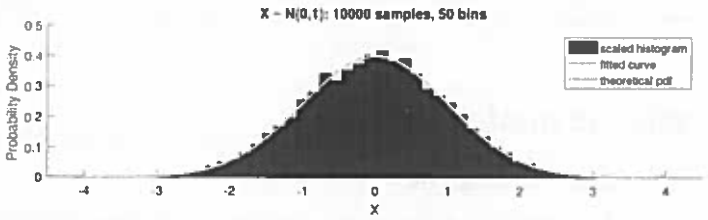


Figure 1.3.1: PDF estimation of Gaussian Distribution

Observation of **figure 1.3.1** shows that the estimation works well for approximating the probability density function of this Gaussian random variable with mean 0, standard deviation 1, and 10000 samples.The fitted curve (red) closely follows the theoretical PDF (green).

Comment on when N increases could be more indepth

Question 2

Stationary and Ergodic Approximation

Good figure captions but legend and axis are too small

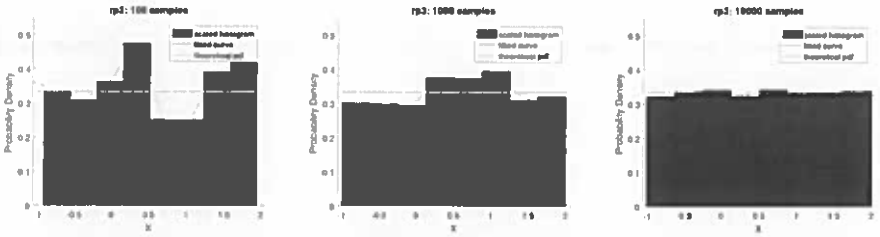


Figure 1.3.2: PDF estimation of rp3 with 10 bins, a stationary and ergodic process.

Figure 1.3.2 shows that as the number of samples increases, the fitted curve (red), and the bins (purple) converge to the theoretical probability density function. This is a confirmation of the previous reasoning in section 1.1, which suggests that the accuracy of a PDF estimate increases as sample number increases.

Correct definition of non-stationary process and sensible solution for computing its pdf

Question 3

Non-Stationary Approximation

The developed function `pdf` calculates an average probability density function across all samples. When a process is non-stationary its probability density function varies over time/samples, and therefore the function cannot accurately estimate the PDF.

In the case of a random Gaussian 1000 sample process that has a mean of 0 for the first 500 samples, and a mean of 1 for the final 500 samples, the overall pdf produced would be bimodal. These peaks would likely be merged in this case considering the fact that the means of 0 and 1 are very close in comparison with the standard deviation. A more accurate interpretation would be to calculate the pdf for the first 500 samples and the last 500 independently, and then display both.

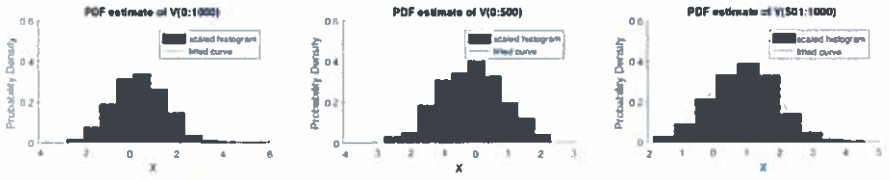


Figure 1.3.3: PDF estimation (10 bins) of stochastic process with mean of 0 for the first 500 samples, and mean of 1 for the last.

From **figure 1.3.3** V(0:1000) we can see a broader and shallower distribution centered at approximately 0.5. For V(0:500) we see a narrower peak centered at approximately 0, and for V(501:1000) we see a narrower peak centered at approximately 1.

2 Linear stochastic modelling

good format of the equations

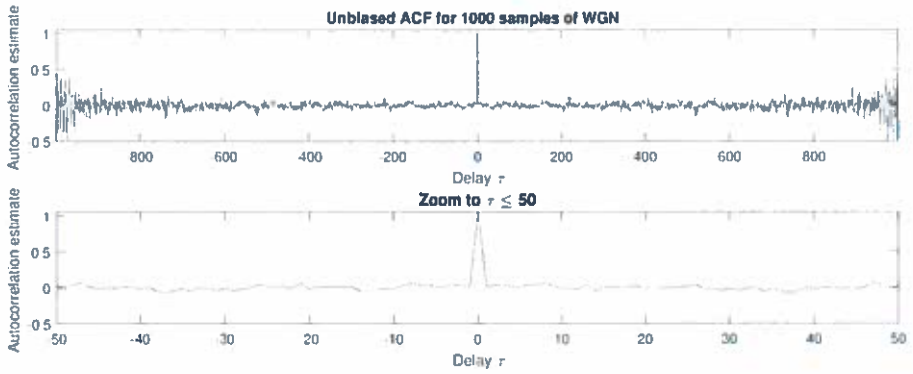
2.1 ACF of uncorrelated sequences

In this subsection we calculate the autocorrelation function of a 1000 sample realisation of white Gaussian noise. This process was also realised previously in **figure 1.1.1b**. Below is the autocorrelation function:

$$R_X(\tau) = \lim_{N\to\infty} \frac{1}{N} \sum_{n=0}^{N-1} x[n]x[n+\tau], \tau \in \mathbb{Z}. \tag{3}$$

We do not have an infinite number of samples in the real world, and therefore we must estimate the autocorrelation function with an unbiased estimator, given by:

$$R_X(\tau) = \frac{1}{N-|\tau|} \sum_{n=0}^{N-|\tau|-1} x[n]x[n+\tau], \tau = -N+1,..,N-1. \tag{4}$$



Correct figures

Figure 2.1.1: An unbiased autocorrelation estimate of a 1000 sample realisation of white Gaussian noise.

Question 1 and 2

In the full ACF estimate in **figure 2.1.1** we see that the autocorrelation is simply a value of 1 for a lag of $\tau = 0$, with small variations around zero for lags of $\tau > 0$. These variations increase in amplitude as τ increases, reaching values of close to and even greater than 1 as $|\tau|$ approaches 1000.
Each sample of white Gaussian noise is independent, and therefore none of the samples are correlated. However when the lag is zero there is a perfect positive correlation, because each sample is being compared with itself. The value of the peak is equal to $E[x^2] = \hat{\sigma}^2 + \hat{\mu}^2$. This gives a theoretical ACF of a dirac delta function (magnitude 1) at $\tau = 0$ and zeros for delays of $\tau \neq 0$.

The autocorrelation estimate is also perfectly symmetrical about $\tau = 0$. This is a general property of autocorrelation, given by:

$$R_X(\tau) = E\left[x[n]x[n+\tau]\right], \quad k = n+\tau$$

Good description of ACF and definition of "white"

Appropriate references to the figures

$$\therefore R_X(\tau) = E\big[x[k-\tau]x[k]\big] = E\big[x[n]x[n-\tau]\big] = R_X(-\tau)$$

The zoom to $|\tau| \leq 50$ (**figure 2.1.1**) displays an ACF much closer to the theoretical function. For delays τ in this range that have a magnitude greater than zero, the autocorrelation estimate is approximately zero.

Question 3

We can see from unbiased estimation equation given previously that as $|\tau|$ increases, the average is taken over $|\tau|$ less samples. Variance of sample mean is proportional to $\frac{\sigma^2}{N}$ where N is the number of samples that the mean is taken from, and σ is the standard deviation. In this case, $\sigma = 1$ and $N = N - |\tau|$. Therefore the standard error is proportional to $\frac{1}{\sqrt{N-|\tau|}}$.

Good explanations and Sensible bounds for lags

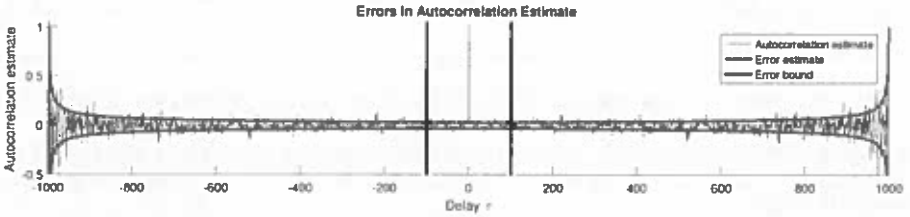


Figure 2.1.2: Predicted error and error bounds compared with ACF estimate.

Figure 2.1.2 shows a plot of the standard error (red) against the autocorrelation estimate (blue). Clearly we can see the sharp increase in error as the lag approaches 1000, which also allows us to note the poor reliability in estimates for large τ. Bounds have been set on τ at $\frac{1}{\sqrt{N-|\tau|}} = 0.033$, giving a 3.3% standard error. This statistically means that approximately 99.7% of samples at this delay will fall within 10% error. Solving the equation gives $|\tau| = 100$ which is also visualised in **figure 2.1.2**.

2.2 ACF of correlated sequences

Question 1

Correct and nice plot

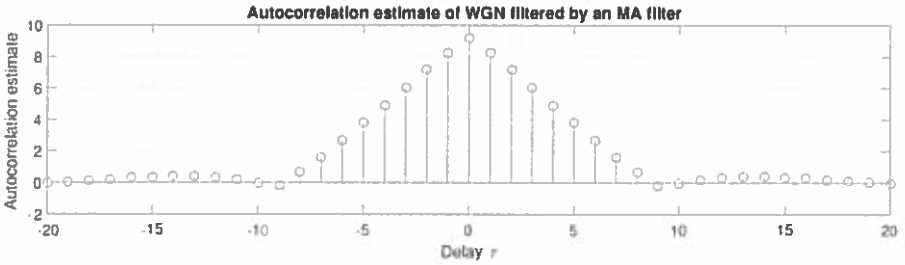


Figure 2.2.1: ACF estimate of 1000 samples of WGN with a moving average of coefficients equal to 1 and order 9.

At the peak, the value is simply the average of $x \cdot x$ which is the expected value of x^2.
The derivation of the value at the peak is given by:

Reasonable explanation

$$\hat{\mu} = (r \cdot \mathcal{O} \cdot \hat{\mu})$$

$$\hat{\sigma} = (r \cdot \mathcal{O} \cdot \frac{\hat{\sigma}}{\sqrt{\mathcal{O}}})$$

$$\therefore E[x^2] = ((r^2 \cdot \mathcal{O}^2 \cdot \hat{\mu}^2) + (r^2 \cdot \mathcal{O} \cdot \hat{\sigma}^2))$$

where $\mathcal{O}$ = order of the filter, r = coefficient value (assuming all coefficients are the same value), $\hat{\mu}$ = sample mean, $\hat{\sigma}$ = sample standard deviation.
In the case of this moving average filter, $r = 1, \mathcal{O} = 9, \hat{\mu}^2 \approx 0$, and $\hat{\sigma}^2 \approx 1$, therefore the peak ends up at approximately 9. The ACF also reduces to the value of $(r^2 \cdot \mathcal{O}^2 \cdot \hat{\mu}^2)$ at a magnitude of delay $|\tau|$ corresponding to the order. This is because at that delay the only relatedness of samples comes from the shared sample mean (the moving average no longer overlaps). In this case that value is approximately zero. Observation of **figure 2.2.1** confirms this theory.

Good figure and nice title and legend

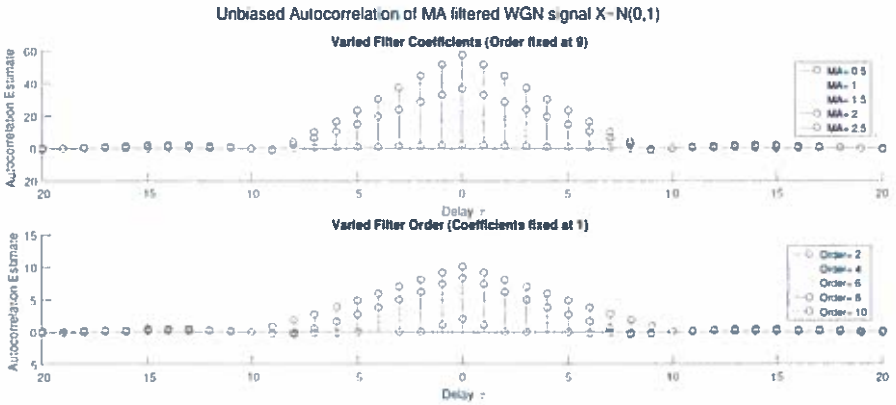


Figure 2.2.2: ACF estimate of 1000 samples of moving average filtered WGN with varying MA coefficients and order.

Figure 2.2.2 indicates that as theorised the peak varies with the coefficients to the power of 2 and the order to the power of 1. We can also observe that as the filter order changes, the ACF estimate always approaches zero at a delay corresponding to that value.

In terms of the precise local sample mean, we simply take the value from our moving average filter, and divide it by the filter order.

Correct explanation of Ry being the impulse response

Question 2

$$R_Y(\tau) = R_X(\tau) * R_h(\tau) \tag{5}$$

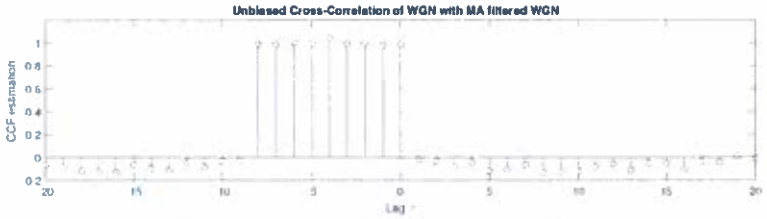
$R_Y(\tau)$ is the autocorrelation of Y_n which is the filtered version of the stochastic process X_n. If X_n is uncorrelated then its ACF is simply a Dirac delta function. The convolution of anything with a Dirac delta function is itself, and therefore $R_Y(\tau)$ represents $R_h(\tau)$ which is the autocorrelation of the impulse response of the filter.

2.3 Cross-correlation function

We now perform cross-correlation of our white Gaussian noise X_n and the moving average filtered version Y_n. The unbiased cross-correlation estimate is given by:

$$\hat{R}_{XY}(\tau) = \frac{1}{N-|\tau|} \sum_{n=0}^{N-|\tau|-1} x[n]y[n+\tau], \tau = -N+1, ..., N-1. \tag{6}$$

Question 1



Correct plot of CCF of MA process

Figure 2.3.1: CCF estimate of 1000 samples of MA filtered ($r = 1, O = 9$) WGN with WGN.

The cross-correlation estimate has peak values clustered around a value of 1 (which is the filter coefficient value), the number of which corresponds to the order of the filter (in this case 9). The symmetry point has also now shifted from zero to a delay of $\tau = -\frac{O+1}{2} = 5$.

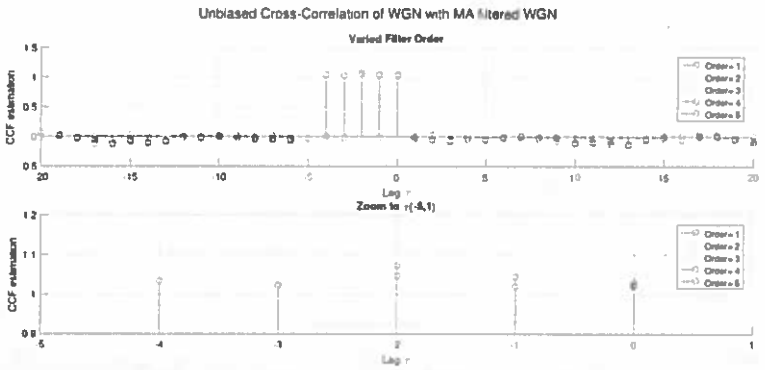
The relation between the input and output is given by:

$$R_{XY}(\tau) = h(\tau) * R_X(\tau) \tag{7}$$

Therefore, if X_t is an uncorrelated stochastic process, it's autocorrelation $R_X(\tau)$ is simply a Dirac delta function. We can therefore say that $R_{XY}(\tau)$ is equal to $h(\tau)$ which represents the impulse response of the filter.

Question 2

Correct answer on how result is used for system ID



Legend could've been bigger

Figure 2.3.2: CCF estimate of 1000 samples of WGN with MA filtered WGN of varied order ($r = 1$).

Observation of **figure 2.3.2** shows the quantity of peak points corresponds to the order of the filter. This is the impulse response of the filter. For system identification purposes, if we input a sequence of WGN ~ N(0,1) into a filter and we perform cross correlation between the output and the input, we will achieve the impulse response. This information can be used to identify a system.

2.4 Autoregressive modelling

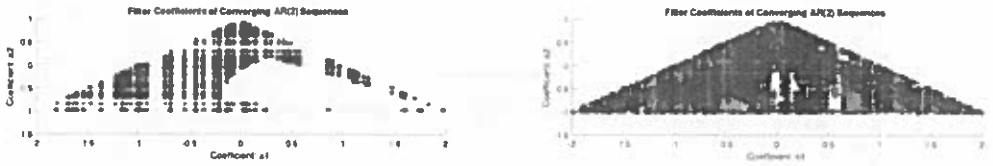
Question 1

An AR(2) process can be defined as:

$$x[n] = a_1 x[n-1] + a_2 x[n-2] + w[n], \quad w[n] \sim N(0,1). \tag{8}$$

Where $w[n]$ is white Gaussian noise, and a_1, a_2 are the filter coefficients.

Correct plot but Axis and title are too small



(a) Coefficients convergence plot for AR(2) process: 1 run **(b)** Coefficients convergence plot for AR(2) process: 8 runs

Figure 2.4.1: Plot of filter coefficients a1, a2, for converging AR(2) processes.

Observation of **figure 2.4.1** shows a stability triangle with vertices at coordinates (-2,1), (0,1), (2,-1), where all pairs of (a_1, a_2) within this triangle produce a stable AR(2) process.

The z-transform of an AR(2) process gives us:

$$z^2 - a_1 z - a_2 z = 0$$

We can solve this equation for the roots:

$$\lambda_1, \lambda_2 = \frac{a_1 \pm \sqrt{{a_1}^2 + 4a_2}}{2}$$

For stability both roots must lie within the unit circle, $|\lambda| < 1$.

$$\frac{a_1 + \sqrt{{a_1}^2 + 4a_2}}{2} < 1 \Rightarrow {a_1}^2 + 4a_2 < (2 - a_2)^2 \Rightarrow \underline{\underline{a_2 + a_1 < 1}} \quad [1]$$

$$\frac{a_1 - \sqrt{{a_1}^2 + 4a_2}}{2} > -1 \Rightarrow {a_1}^2 + 4a_2 < (2 + a_2)^2 \Rightarrow \underline{\underline{a_2 - a_1 < 1}} \quad [2]$$

For complex roots: $a_1^2 + 4a_2 < 0$

$$\lambda_1, \lambda_2 = \frac{a_1 \pm i\sqrt{a_1^2 + 4a_2}}{2} \Rightarrow \lambda^2 = \frac{a_1^2}{4} - \frac{a_1^2 + 4a_2}{4} = -a_2$$

$$\therefore |\lambda| < 1 \Rightarrow \underline{\underline{-1 < a_2 < 1}} \quad [3]$$

Proof of stability is presented and correct

Stability conditions 1, 2, and 3 combine to give us the stability triangle shown in **figure 2.4.1**.

Question 2

Correct plots of ACF for sunspot of different length

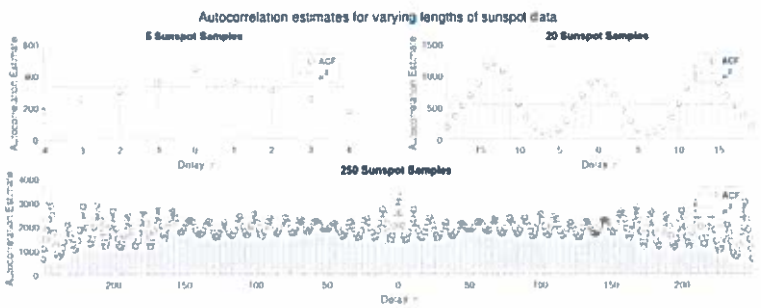


Figure 2.4.2: Autocorrelation estimates for varying lengths of sunspot count data.

Observation **figure 2.4.2** shows us that as the number of samples of sunspot data increases, so does the number of peaks of its autocorrelation. The autocorrelation of the data exploits the periodicity in sunspot behavior, and therefore by increasing the time span of sample data we increase the number of periods that we are visualising. These plots are also compared with the individual sample means to the power of two, (μ^2). This value shows how the data is offset by a value that is approximately μ^2, and observation of **figure 2.4.3** (ACF of zero mean data) confirms this.

Good explanation of the effect of the mean

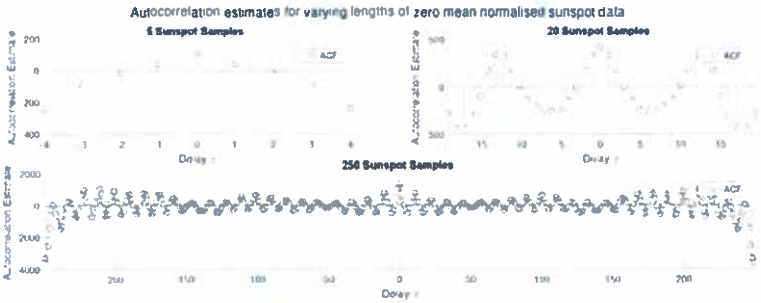


Figure 2.4.3: Autocorrelation estimates for varying lengths of zero mean normalised sunspot count data.

Question 3

Using the Yule-Walker equations through MATLAB function `aryule`, calculation has been made of the partial correlation functions for sunspot data that is standardised to zero mean and unit variance, and raw sunspot data.

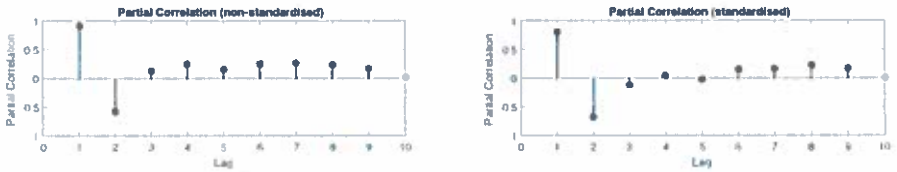


Figure 2.4.4: Partial correlation coefficients for sunspot data using Yule Walker equations.

Correct identification of model order

From comparison of **figure 2.4.4**, the partial correlation coefficients for standardised and non standardised data are similar, but the coefficients for the non standardised coefficients remain significantly higher than the standardised coefficients for lags of 3,4 and 5. This is because the mean of the data provides added correlation between each data point.
In both models, the coefficient magnitude drops significantly after a lag of 2, suggesting that an AR(2) model order is likely optimal. Depending on what is classified as a large coefficient size, an argument can also be made for AR(9) as after this lag the coefficient magnitude falls significantly lower again.

Question 4

Minimum description length (MLD) and Akaike information criterion (AIC) can be used to determine optimal model order for standardised data. They weight the error with a penalty for increased model order (due to increased computational complexity).

$$MDL = logE_p + \frac{plogN}{N} \tag{9}$$

$$AIC = logE_p + \frac{2p}{N} \tag{10}$$

- E_p is the loss function. In this case the cumulative squared error.
- p is the model order.
- N is the number of data points.

Explanations could've been more indepth

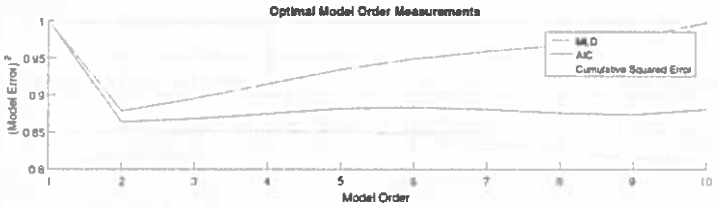


Figure 2.4.5: Minimum description length and Akaike information criterion for standardised data.

From observation of **figure 2.4.5** we can see that an order of 2 minimises both the MLD and the AIC, and is therefore our optimal model order. The AR(9) model may have less error, but it introduces far too much computational complexity and this makes it a poor model.

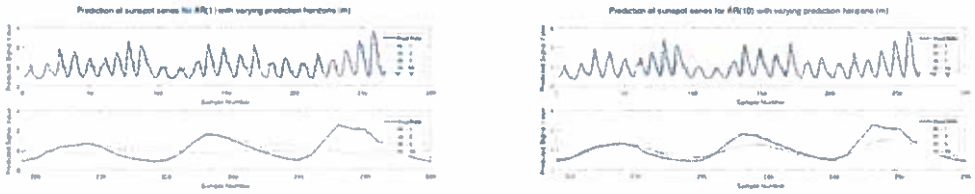
Question 5

AR models can be used to perform data predictions m steps ahead, with the equation:

$$\hat{x}[n+m] = a_1 x[n-1] + \ldots + a_p x[n-p]$$

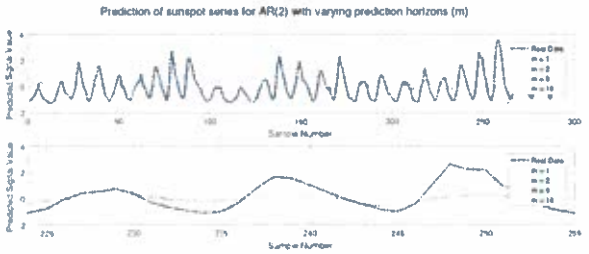
where m is the prediction horizon, and p is the model order.

Figure 2.4.6 shows a reduction in accuracy of estimate as the prediction horizon increases, and an increase in accuracy of estimate with an increase in model order. These observations make logical sense, as a higher order model takes into account more data for each prediction and is therefore likely to be more accurate. Also, as the prediction horizon increases then there is more time since the last real data point, and the prediction will likely be less accurate. The mean squared error plot in **figure 2.4.7** confirms these observations.
The mean and variance comparisons of **figure 2.4.7** show that in general the mean and variance tends to zero as prediction horizon increases. This is because as the prediction horizon increases, the model is further from the last measured data point, and therefore the data influences it less. The prediction becomes shallower in amplitude and much closer to zero.
For a prediction horizon of 1 and 2, the AR(10) model has a mean that is further away from the theoretical value of 0 than the AR(2) model. This is likely due to the over-fitting of AR(10) which causes it to fluctuate far more around the expected value, and spike a lot higher. This also explains the distinctly higher variance of the AR(10) model, although this is closer to the theoretical value of 1.



(a) AR(1)

(b) AR(10)



title, legend and axis are too small

(c) AR(2)

Figure 2.4.6: Prediction of sunspot series compared with prediction horizons for different order AR models.

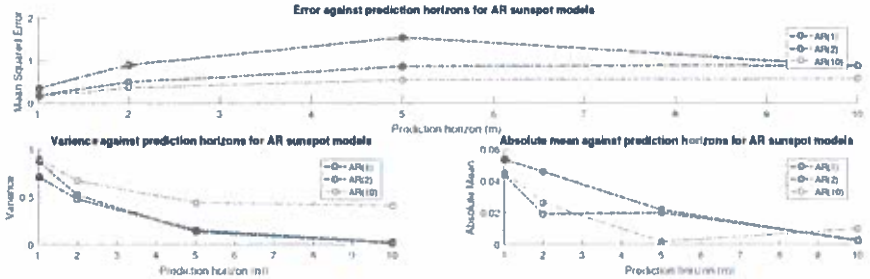


Figure 2.4.7: Statistical properties of AR models against prediction horizons (taken from samples 50:288 in order to exclude any transient phase of the filter).

2.5 Real world signals: ECG from iAmp experiment

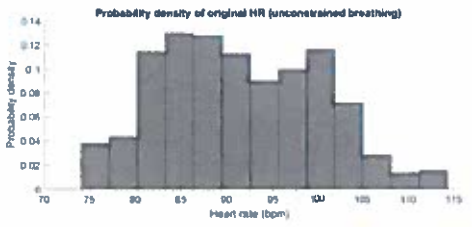
Question 1

By extracting RR interval data (time interval between heart beats) we can achieve a local heart rate in beats per minute.

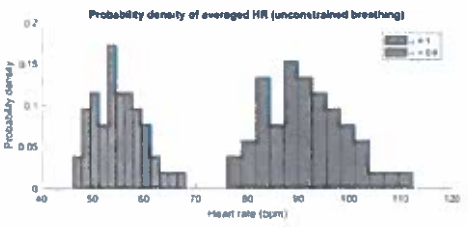
$$h[n] = \frac{60}{rr[n]} \tag{11}$$

We can also achieve a smoother estimate by averaging every 10 samples of heart rate.

$$\hat{h}[n] = \frac{1}{10} \sum_{i=(10(n-1)+1}^{10n} ah[i] \tag{12}$$



(a) PDE for original heart rate



(b) PDE for averaged heart rate

Figure 2.5.1: Probability density (normalised by sum) for heart rate during unconstrained breathing.

Correct plots but legend, title and axis are small

Observation of **figure 2.5.1** shows that in general the heart rate follows a roughly Gaussian distribution that has a slight positive skew, with a peak at approximately 91.5bpm. It also shows that the spread of values is larger for the original heart rate calculations. This is because the average reduces the impact of outliers in the data so that they aren't significant in the probability density estimate. The introduction of $\alpha = 0.6$ simply scales all of the values in the x axis, and the bar width over which probability is evaluated. This introduces a bias of 0.6μ in the mean. Some data points are shifted into different bins which leads to the bar height varying slightly.

Averaging reduces the variance of the signal:

$$var(\hat{h}) = \frac{1}{10^2}\sum_{i=1}^{10} var(\hat{h}_i)$$

Using the assumption that the data points are uncorrelated:

$$var(\hat{h}) = \frac{10}{10^2} = \frac{1}{10}$$

Therefore averaging as we did above with $\alpha = 1$ reduces the variance 10 fold, hence why we observe less spread in the data and the probability density estimate is compressed.

Question 2

AR modelling of heart rate:

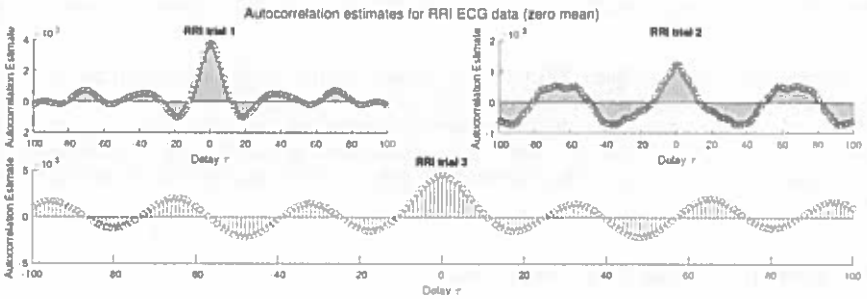


Figure 2.5.2: ACF estimates for different trials of RRI ECG data.

Figure 2.5.2 (especially trial 1) shows an autocorrelation that decays with a gradual exponential, similar to the sunspot data's autocorrelation (an AR process) in part 2.4. We know from part 2.2 that the autocorrelation of an zero mean MA process decays to zero at the filters order. From these observations we can infer that the RRI data is likely an autoregressive process.

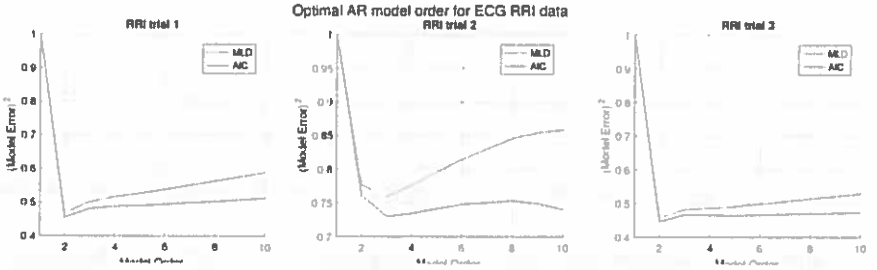


Figure 2.5.3: ACF estimates for different trials of RRI ECG data.

Observation of **figure 2.5.3** shows that for **trial 1** and **trial 2**, the minimum description length and Akaike information criterion suggest an optimal model order of AR(2). For **trial 2** the MDL and AIC suggest AR(3). This is likely due to a faster breathing rate introducing more variability into the data for trial 2, which is optimally fitted by a higher model order. Because of trial 1 and 2, We can infer that an AR(2) model for RRI ECG data is optimal.

Good explanation but a link to the term 'bias' hasn't been made

3 Spectral estimation and modelling

We can estimate the power spectral density of a signal by basing a periodogram of the signal on the fast Fourier transform, as defined by:

$$\hat{P}_x(f) = \frac{1}{N}\left[\sum_{n=0}^{N-1} x[n]e^{-j2\pi f \frac{n}{N}}\right] \tag{13}$$

The FFT estimate of power spectral density is implemented in a matlab function called `pgm`, which is used to estimate the PSD for WGN in **figure 3.0.1**.

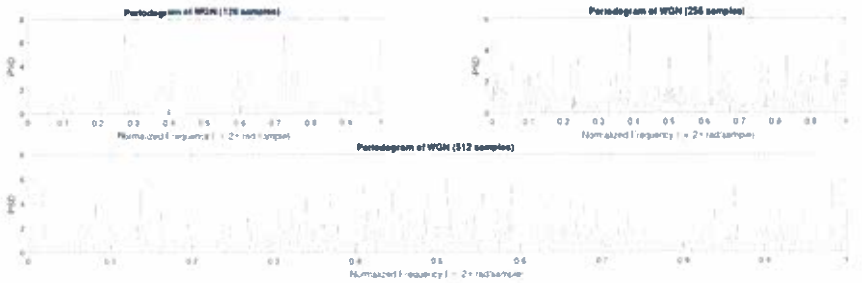


Figure 3.0.1: Power spectral density estimates for different sample lengths of white Gaussian noise ~ $N(0,1)$

We can see through observation of **figure 3.0.1** that the power spectral density is symmetrical about $\pi rad/sample$. As the sample number increases, the periodogram becomes denser as there are more frequency points over which the magnitude is evaluated. The expected magnitude response of white noise should be flat, as by definition it should contain equal proportions of every frequency. However, in practice the random nature of white noise does not produce a power density spectrum that is flat. Rather, it produces a power density spectrum that is itself the square of white Gaussian noise.

3.1 Averaged periodogram estimates

Question 1

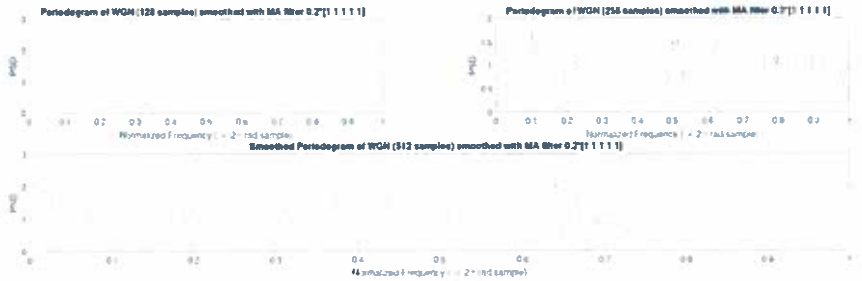


Figure 3.1.1: Power spectral density estimates for different sample lengths of white Gaussian noise ~ $N(0,1)$

Observation of **figure 3.1.1** shows a decrease in the range of the amplitudes of the PSD, as the MA filter has smoothed out the sharp spikes in the data. However, the power spectral density is also shifted forward by 2 samples because of the filter delay. The obvious solution to eliminate this delay would be simply shifting the spectrum back by a value corresponding to the delay.

Question 2 and 3

Observation of **figure 3.1.2** shows vast changes in the mean, around the theoretical value of 1, and standard deviation between the power spectral density for 8 different segments of a white Gaussian noise sample. In section 1.1 we showed variability in WGN sample lengths of 1000. In this case the sample length is only 128 for each segment, which explains the large variability.

Correct plots

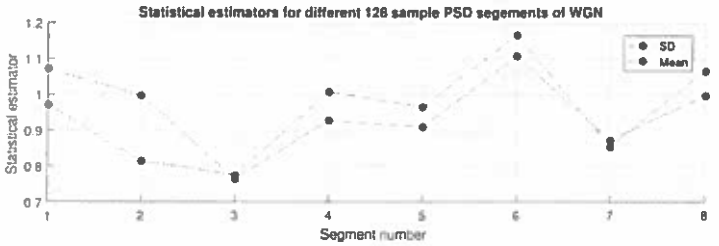


Figure 3.1.2: Mean and standard deviation of the PSDs for 8 different segments of WGN.

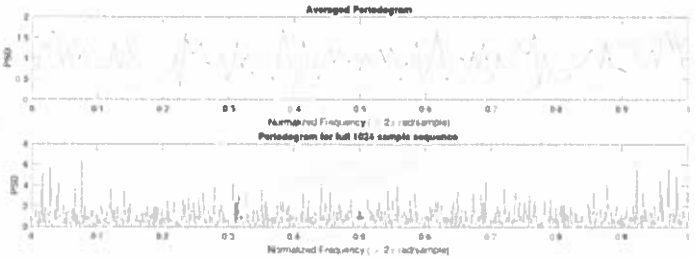


Figure 3.1.3: Average of the PSDs of each segment, compared with the PSD of the whole 1024 sample sequence.

Figure 3.1.3 displays the averaged PSD across all segments above the power spectral density of the full 1024 sample WGN. We can see that the averaged PSD is simply a smoothed version of the full PSD. This is a similar result to an MA filtered PSD, but with no delay.

Sufficient number of periodograms but the title is small

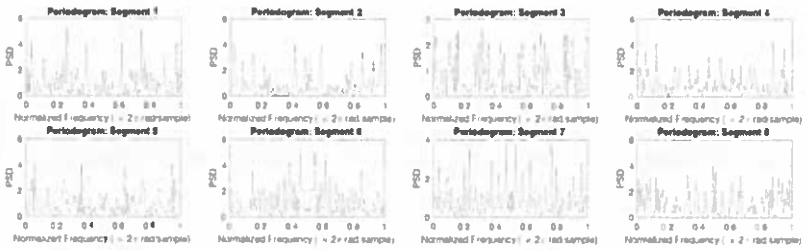


Figure 3.1.4: Individual periodograms for each of the 8 128 sample segments.

Observation of **figure 3.1.4** shows that each individual PSD is very different from the overall estimate. However, it is clear that each segment has a distinct contribution to the PSD of the whole 1024 sample sequence.

3.2 Spectrum of autoregressive processes

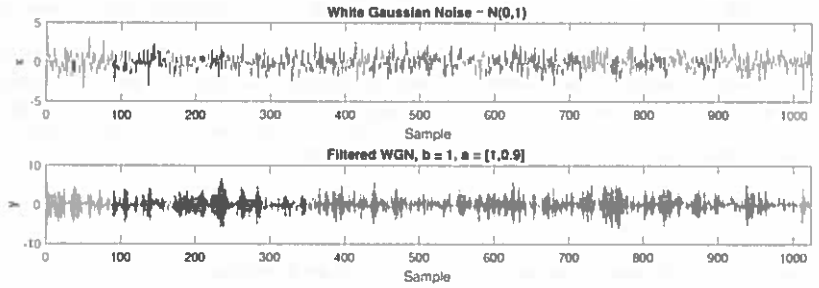


Figure 3.2.1: White gaussian noise filtered by an AR filter: b = 1, a = [1,0.9].

Figure 3.2.1 appears to show a high pass filtered signal. The characteristic dense bursts of signal y show that higher frequency aspects of the signal have increased in magnitude significantly, whereas the lower frequency parts have been

removed.

The theoretical PSD of an AR(p) process is given by:

$$\hat{P}_Y(f) = \frac{\sigma_X^2}{\left|1 + \sum_{k=1}^{p} a_k e^{-j2\pi k f}\right|^2} \tag{14}$$

With an AR(1) model with coefficient 0.9, and variance $\sigma_x^2 = 1$, we can rewrite this as:

$$\hat{P}_Y(f) = \frac{1}{\left|1 + 0.9e^{-j2\pi f}\right|^2} \tag{15}$$

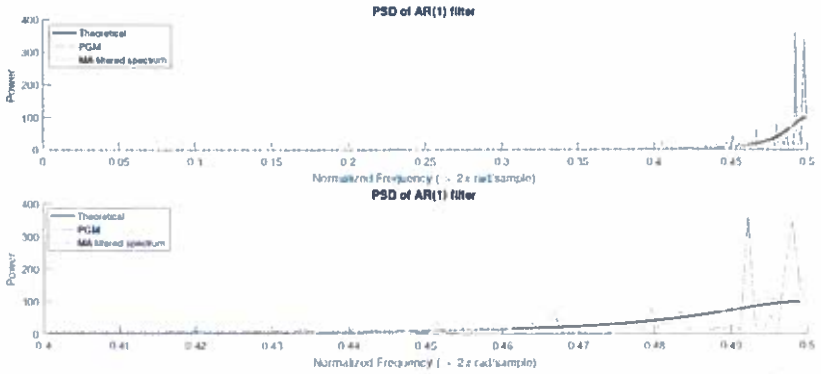


Figure 3.2.2: Power spectral density estimation with data (PGM), smoothed PGM, and theoretical prediction.

Question 1

In technical terms, the cut off frequency (-3dB magnitude) occurs at a frequency of 1.47 radians per second. Observation of **figure 3.2.2** shows that realistically, considering the signal to noise ratio, any portion of the signal falling below $(0.4) \cdot (2\pi) = 2.5 rad/s$ is irrelevant. The shape of the response is a bell-shaped curve centered at $\pi rad/s$, indicating a high pass filter.

Good explanation of high pass nature

Question 2

The result of our estimation function (shown as **PGM** in **figure 3.2.2**) is a very spikey spectrum that follows the same overall trend as the theoretical estimate. The large variance in the spectrum occurs due to the finite sample size, but I have reduced this effect by also smoothing the spectral estimate. The MA filtered spectrum is observed to follow the theoretical spectrum very closely.

Question 3

Observation of **PGM** in **figure 3.2.2** shows that the main peaks are left of where they should be, with a dip in the spectrum where the peak should actually be. This not only because of the finite sample size, but also the **windowing effect**.
Our spectral estimation is based on the fast Fourier transform of our data. We do not have an infinite number of samples/an infinite time duration to observe the noise over. Therefore, to take the FFT we must use a form of windowing to isolate our sample. A rectangular window is a sinc function in the frequency domain. Sinc functions have side lobes of which the magnitude decays exponentially, and therefore when the frequency spectrum is convolved with a sinc function leakage occurs that spreads out the frequencies to a certain extent.

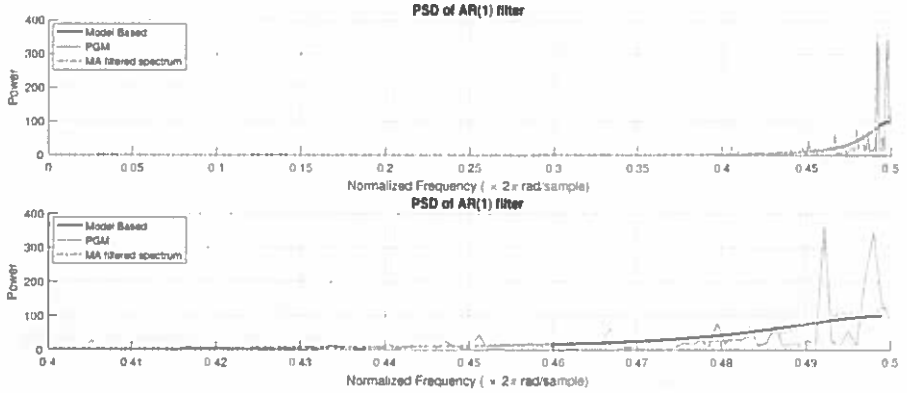
Question 4

Correct explanation including the windowing and how it causes high freq. artefacts

We can estimate the variance and the coefficient of an AR(1) model by extracting the zeroth and first coefficients from the autocorrelation estimate of the data, and using the following equations:

$$\hat{a}_1 = -\hat{R}_Y(1)/\hat{R}_Y(0) \tag{16}$$

$$\hat{\sigma}_X^2 = \hat{R}_Y(0) + \hat{a}_1 \hat{R}_Y(1) \tag{17}$$



Correct plots of the AR spectrum estimate and periodogram estimate

Figure 3.2.3: Power spectral density estimation with data (PGM), smoothed PGM, and model based prediction.

Observation of **figure 3.2.3** shows that the model based PSD follows the trend of the data's spectrum almost perfectly, and is very close to the theoretical spectrum shown in **figure 3.2.2**.

Question 5

Sunspot model based PSD:
By using `ARYULE` to estimate our sunspot coefficients in the same way that we estimated our AR(1) model above, for other orders, we can calculate a model based PSD for our sunspot data.

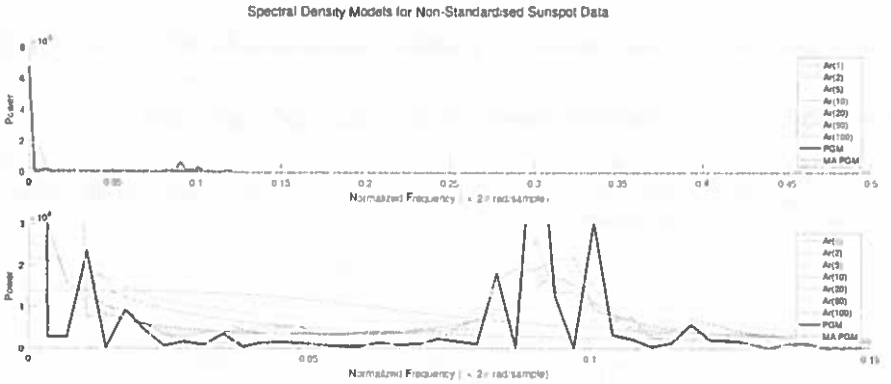


Figure 3.2.4: Power spectral density estimation of non-standardised sunspot data, for AR models 1,2,5,10,20,50,100.

Observation of **figure 3.2.4 and 3.2.5** shows that the spectral density estimate of the sunspot data (both standardised and original) has a peak at $0.01 \cdot (2\pi) \approx 0.063 rads/sample$, and a peak at $0.09 \cdot (2\pi) \approx 0.57 rads/sample$. The effects of under modelling are clear from AR(1), which completely misses the second peak in its modelling. The effects of over modelling are also clear in AR(100), which exhibits peak splitting on both the first and second peaks, and oscillates heavily between the peaks. From this scale there is no evidence of over fitting associated with a model order of 10.
In **figure 3.2.6** the effects of over modelling are now also obvious for the AR 10 model. Our sunspot data follows an AR 2 model with 1 important peak in the spectrum, but the over fitting of the AR 10 model predicts multiple peaks in the frequency spectrum.

the effects of under and over modelling on spectrum is well explained

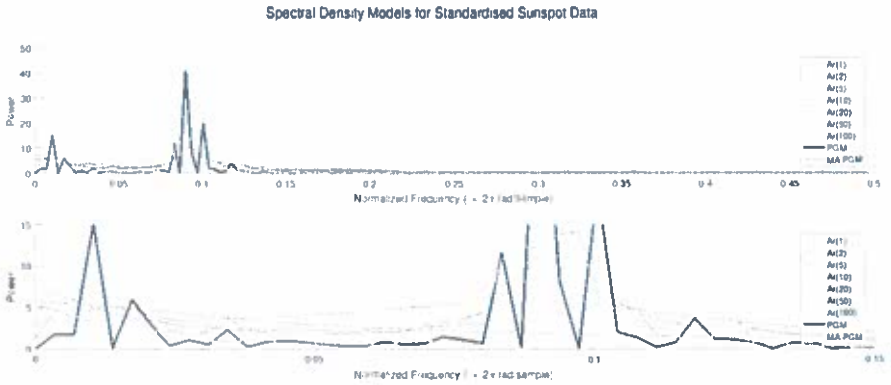


Figure 3.2.5: Power spectral density estimation of standardised sunspot data, for AR models 1,2,5,10,20,50,100.

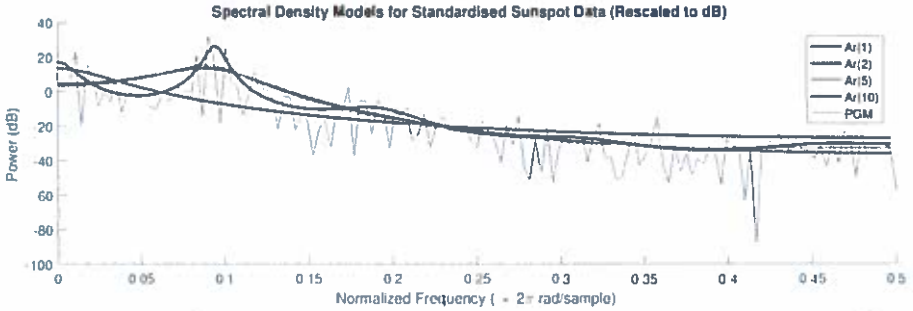


Figure 3.2.6: Power spectral density estimation of standardised sunspot data, for AR models 1,2,5,10, with logarithmic power scale (dB).

3.3 Spectrogram for time-frequency analysis: dial tone pad

Each digit of a telephone number is made up two sinusoids of different frequencies (**table 3.3.1**) in the voice frequency band, which distinguish that number. This is the Dual Tone Multi-Frequency telecommunications system. Each signal is generated through the following formula:

$$y[n] = sin(2\pi f_1 n) + sin(2\pi f_2 n) \tag{18}$$

	1209 Hz	*1336 Hz*	*1477 Hz*
697 Hz	1	2	3
770 Hz	4	5	6
852 Hz	7	8	9
941 Hz	*	0	#

Table 3.3.1: Dial pad frequencies for Dual Tone Multi-Frequency system.

Question 1

In **figure 3.3.1** we can clearly see a difference between the 4th and 5th frequencies. We also observe the noiseless 0.25 second delay a flat signal between each digit.
The sampling frequency is 32768. This satisfies the Nyquist sampling criterion of being greater than twice the maximum frequency of 1477Hz. Furthermore, it allows each dial tone to be made up of 8192 samples, which corresponds to 2^{13}, and this allows the Radix-2 FFT algorithm to compute more efficiently with $\frac{N}{2}\log_2 N$ complex multiplications rather than N^2 complex multiplications. Finally, telephone lines are very long and therefore are subject to a lot of external noise. In practice, oversampling can increase our resolution and decrease the impact of noise, allowing us to observe the underlying frequencies regardless of a poor signal to noise ratio (*This will be shown in part 4*).

Correct time domain plot of the phone number

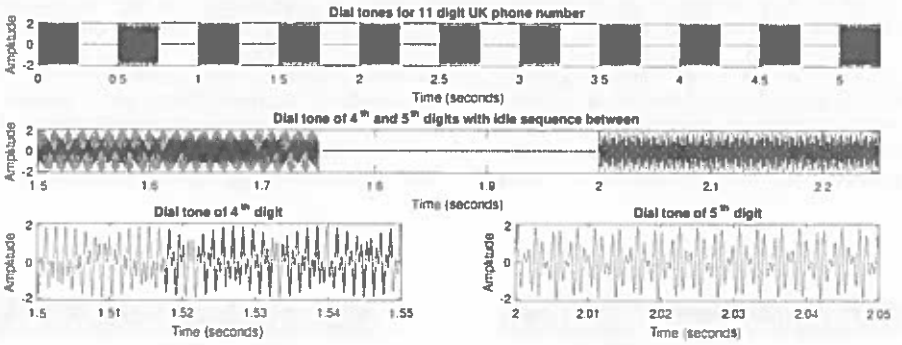


Figure 3.3.1: Dial tone sequence for 11 digit UK phone number

Question 2

Nice and correct spectrogram plots

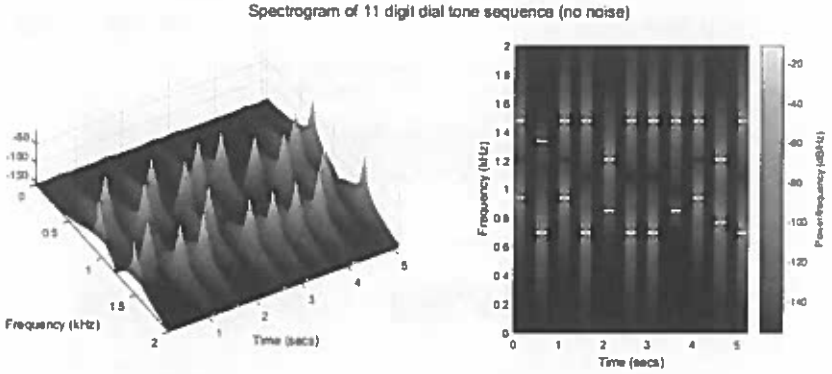


Figure 3.3.2: Spectrogram of 11 digit UK phone number dial tone sequence with zero noise.

The spectrogram is calculated using an 8192 sample width Hanning window with zero overlap, corresponding to the sample width of each dial tone/idle sequence (each 0.25 second section). We can observe in **figure 3.3.2** two frequency spikes corresponding to each dial tone. Comparison with **table 3.3.1** yields the phone number *02037339043* which is indeed the number that was generated.

Correct explanation of key classification using spectrogram and table

Question 3

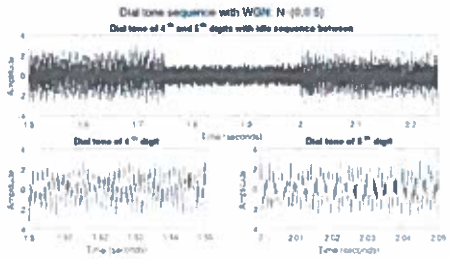
We can clearly identify from **figure 3.3.2** the dual frequencies that make up each digit, as they occur as obvious peaks on the spectrum. A simple code which measures the frequencies corresponding to each peak and relates them to **table 3.3.1**, will be sufficient to perform key classification. In MATLAB we can simply extract the spectrogram sequence from the function `spectrogram` and then set a threshold value for identification of peaks. We can then have a series of if statements which check if the peak falls within a small range corresponding to each dial tone frequency, and assign a numerical value depending on which dial tone frequency the peak is at. Each tone will end up with two numerical values for its two peaks, and these will be used as the row and column references for an array containing the phone digit. This code will loop through the spectrogram until it has extracted all 11 digits.

Question 4

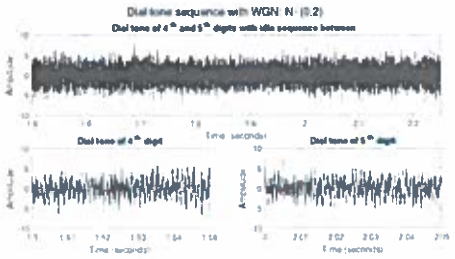
Observation of **figure 3.3.3** shows that time domain classification quickly becomes impossible after the addition of white noise. With white noise that has a variance of 0.5, we can still make out a difference between the dial tones for individual digits, and the idle time between each digit. However, as the variance increases to 2, the differences between each dial tone sequence become unrecognisable in the time domain, and even the idle time can no longer be observed. When the variance increases further to 5, the dial tone sequence simply looks like noise, with no recognisable underlying signal.

identification problem using the minimum frequency difference between 2 keys is well explained

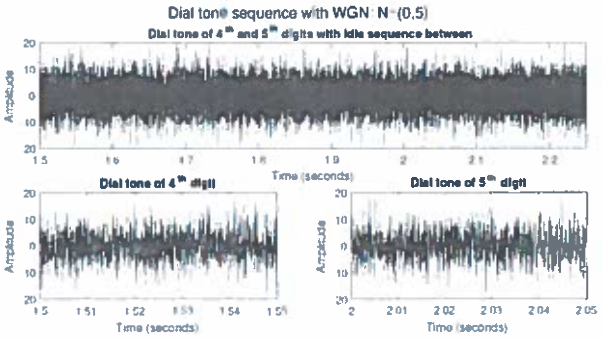
Despite this, we can still observe two peaks corresponding to each digit if we observe the spectrogram. **Figure 3.3.4** shows that even with the addition of WGN that has a variance of 5, there are still two obvious peaks that can be associated with each digit. On the 3 dimensional view of the spectrum, we can see that each peak still protrudes significantly above the rest of the signal frequencies. In the frequency domain, the addition of white noise simply means that every single frequency has more magnitude associated with it. On the spectrogram we observe this as the single 'floor' level rising as the variance of the noise increases. It is obvious that there will be a point at which this level would become higher than the dial tone peaks, however even a noise with a variance of 5 compared with a underlying signal amplitude of 2 gives an unrealistically poor signal to noise ratio. Practically the noise level will never get close to this.



(a) WGN: N~(0,0.5)

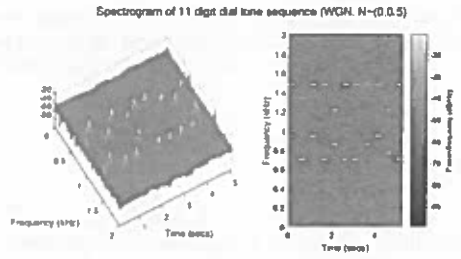


(b) WGN: N~(0,2)

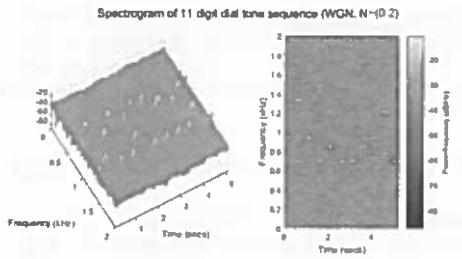


(c) WGN: N~(0,5)

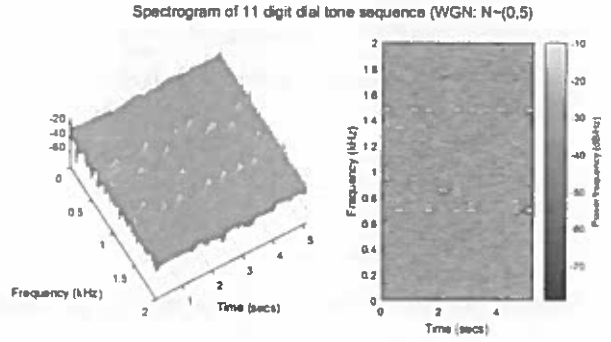
Figure 3.3.3: Dial tone sequences with different levels of white Gaussian noise.



(a) WGN: N~(0,0.5)



(b) WGN: N~(0,2)



(c) WGN: N~(0,5)

Figure 3.3.4: Dial tone sequences with different levels of white Gaussian noise.

3.4 Real World Signals: Respiratory sinus arrhythmia from RR-Intervals

Respiratory sinus arrhythmia is the process in which respiration modulates the heart rate. We see an increase in heart rate upon inspiration, and a decrease in heart rate upon expiration.

Correct identification of breathing rate from peaks from the periodogram

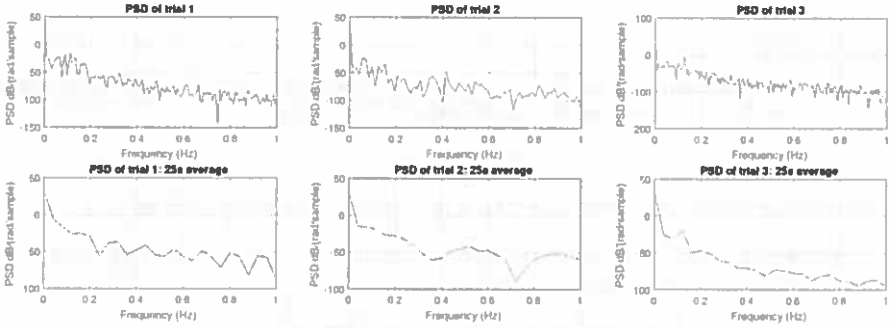


Figure 3.4.1: Periodograms of the different trials of RRI heart rate data, both standard and ensemble averaged with a 25 second window length.

Trial 2 was performed with a respiration rate of 25 breaths per minute, trial 3 was performed with a respiration rate of 7.5 breaths per minute, and trial 1 had no set respiration rate. Therefore, for trial 1 we would expect no discernible set peak in the periodogram, for trial 2 we would expect to find a peak at $\frac{25}{60} = 0.417Hz$, and for trial 3 we would expect to find a peak at $\frac{7.5}{60} = 0.125Hz$.

Close observation of **figure 3.4.1** yields a peak at $0.4152Hz$ for trial 2, and a peak at exactly $0.125Hz$ for trial 3. These results are in line with our expectations. Furthermore, we see a peak at $0.32Hz$ for trial 1, suggesting that our subjects comfortable breathing rate at that moment was approximately 19 breaths per minute. Average breathing rate at rest for a healthy adult is 12-16 breaths per minute[2], which suggests that our subjects breathing rate at the time was high. However, this is in accordance with the earlier heart rate measurements which showed a mean heart rate of 91.5bpm, which is also high. The conclusion is that since the subject displayed no signs of being unwell, the act of

[2] www.hopkinsmedicine.org 21/03/2017

taking measurements from them likely increased their level of stress and therefore breathing rate and respiration rate. Further observation of the averaged periodograms shows clearly the second harmonics associated with the respiration rates of trial 2 and 3. The averaged periodograms show more peaks at around $0.84Hz$, and $0.25Hz$ for trial 2 and 3 respectively, which correspond to twice the expected frequencies.

4 Optimal filtering - fixed and adaptive

Adaptive filtering allows us to characterize an unknown system in terms of filter coefficients, by changing the weights of the coefficients in the adaptive filter in such a way that minimizes the error between the adaptive response and the response of the unknown system.

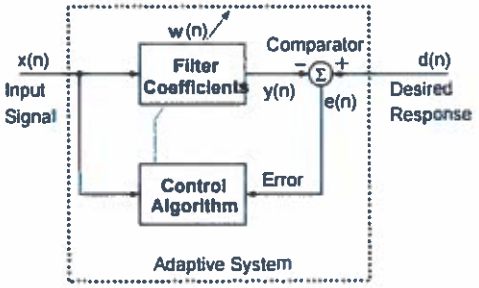


Figure 4.0.1: Adaptive filter for system identification (the diagonal arrow dictates an adaptive filter) [3]

Definitions:

Throughout this section the terms overshoot and rise time will be used when referring to filter coefficient values. Overshoot has been defined as the maximum overshoot (%) in proportion to the filter coefficient, across all coefficients. Rise time has been defined as the time it takes (sample number) for a coefficient to reach 95% of its true value, and in this case the maximum rise time is always taken. (It is given as zero if one or more of the coefficients fail to reach 95% of their true value).

4.1 Wiener filter

The aim of a Wiener filter is to determine an optimum set of weights which minimize the difference between a desired response and a system output. The input output relation of **figure 4.0.1** is given by:

$$y(n) = \sum_{k=1}^{p} w_k x_k(n)$$

where w_k describes the individual filter weights. The error that is to be minimized is given by:

$$e(n) = d(n) - y(n)$$

where $d(n)$ is the desired response, that should be as close as possible to $y(n)$. One may take it a step further, and decide to minimize the mean squared error or error power, J.

$$J = \frac{1}{2}E\{e^2(n)\} = \frac{1}{2}E\{d(n) - y(n)\} = \frac{1}{2}E\{(d(n) - \sum_{k=1}^{p} w_k x_k(n))^2\}$$

The Wiener filter determines the optimal set of filter weights w for which the error power is minimized.

$$J = \frac{1}{2}E\{d^2\} - E\Big\{\sum_{k=1}^{p} w_k x_k d\Big\} + \frac{1}{2}E\Big\{\sum_{k=1}^{p}\sum_{j=1}^{p} w_k x_k w_j x_j\Big\}$$

$$J = \frac{1}{2}E\{d^2\} - \sum_{k=1}^{p} w_k E\{x_k d\} + \frac{1}{2}\sum_{k=1}^{p}\sum_{j=1}^{p} w_k w_j E\{x_k x_j\}$$

The power of the desired signal is $\sigma_d^2 = E\{d^2\}$, the cross correlation between the input and desired response is $r_{dx}(k) = E\{dx_k\}$, and the autocorrelation of the input is defined as $r_{xx}(j,k) = E\{x_j x_k\}$.

$$\therefore J = \frac{1}{2}\sigma_d^2 - \sum_{k=1}^{p} w_k r_{dx}(k) + \frac{1}{2}\sum_{k=1}^{p}\sum_{j=1}^{p} w_k w_j r_{xx}(j,k) \tag{19}$$

[3] D.P. Mandic: Advanced Signal Processing lecture 7, slide 6, 2017

To find the optimal values of w_k that minimize **equation 19**, the derivative of J is calculated wrt w_k and set to zero.

$$\frac{\partial J}{\partial w_k} = 0 = -r_{dx}(k) + \sum_{j=1}^{p} w_j r_{xx}(j,k)$$

$$\sum_{j=1}^{p} w_j r_{xx}(j,k) = r_{dx}(k), \quad k = 1, 2, \ldots, p$$

$$\therefore w_{opt} = r_{xx}^{-1} r_{dx}$$

In vector form:

$$\mathbf{w_{opt}} = \mathbf{R_{xx}^{-1}} \mathbf{p_{dx}}$$

Figure 4.1.1: Wiener-Hopf equation for minimizing error power, in vector form.

Where:

$$\mathbf{p_{dx}} = \begin{bmatrix} r_{dx}(0) \\ r_{dx}(-1) \\ \vdots \\ r_{dx}(-N_w) \end{bmatrix} \quad \mathbf{R_{xx}} = \begin{bmatrix} r_{xx}(0) & r_{xx}(-1) & \cdots & r_{xx}(-N_w) \\ r_{xx}(1) & r_{xx}(0) & \cdots & r_{xx}(-N_w+1) \\ \vdots & \vdots & \ddots & \vdots \\ r_{xx}(N_w) & r_{xx}(N_w-1) & \cdots & r_{xx}(0) \end{bmatrix}$$

N_w corresponds to the filter order minus 1.

Question 1

The unknown system is a filter with coefficients $\mathbf{b} = [1, 2, 3, 2, 1], \mathbf{a} = [1]$. The input to the filter is 1000 samples of WGN $x[n]$ with $\sigma^2 = 1$. The measured output $z[n]$ is the true output $y[n]$ with added WGN $\eta[n]$ of variance $\sigma^2 = 0.1$, as this aims to represent limitations of output measurements in the real world ($z[n] = y[n] + \eta[n]$). The error to be minimized is $e[n] = z[n] - \hat{y}[n]$, where $\hat{y}[n]$ is the output of the Weiner filter.

$$\mathbf{p_{zx}} = \begin{bmatrix} 1.0072 \\ 1.9346 \\ 2.9231 \\ 1.9291 \\ 1.0019 \end{bmatrix} \quad \mathbf{R_{xx}} = \begin{bmatrix} 0.9595 & -0.0033 & 0.0327 & -0.0328 & 0.0198 \\ -0.0033 & 0.9595 & -0.0033 & 0.0327 & -0.0328 \\ 0.0327 & -0.0033 & 0.9595 & -0.0033 & 0.0327 \\ -0.0328 & 0.0327 & -0.0033 & 0.9595 & -0.0033 \\ 0.0198 & -0.0328 & 0.0327 & -0.0033 & 0.9595 \end{bmatrix}$$

Correct optimal coefficients

$$\mathbf{w_{opt}} = \begin{bmatrix} 1.0022 \\ 1.9964 \\ 2.9921 \\ 1.9906 \\ 0.9968 \end{bmatrix}$$

Figure 4.1.2: Optimal coefficients $\mathbf{w_{opt}}$ of the Wiener filter, no scaling in unknown system output $y[n]$. ($SNR(z[n]) = 32.7dB$)

Summing the squared error between the non-scaled $\mathbf{w_{opt}}$ and the unknown system coefficients $\mathbf{b} = [1, 2, 3, 2, 1]$, yields an summed error power of $1.7852e-04$, representing close similarity between the two sets.

$$\mathbf{w_{opt}} = \begin{bmatrix} 0.2350 \\ 0.4678 \\ 0.6988 \\ 0.4637 \\ 0.2346 \end{bmatrix}$$

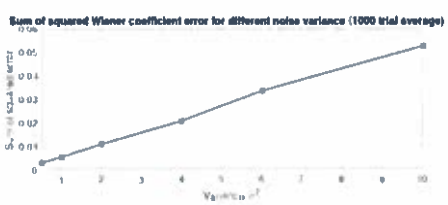
Figure 4.1.3: Optimal coefficients $\mathbf{w_{opt}}$ of the Wiener filter, system output $y[n]$ variance σ^2 scaled to unity. ($SNR(z[n]) = 20.1dB$)

In the case of $\mathbf{w_{opt}}$ produced by a scaled output, $\mathbf{p_{zx}}$ is scaled by approximately the same factor, leading to $\mathbf{w_{opt}}$ also scaled by approximately the same factor. The optimal coefficients also maintain approximately the same ratio independent of scaling. This isn't exact because the variance of $\eta[n]$ is now larger in proportion to the output signal, and therefore there is a reduced signal to noise ratio, leading to further inaccuracy.

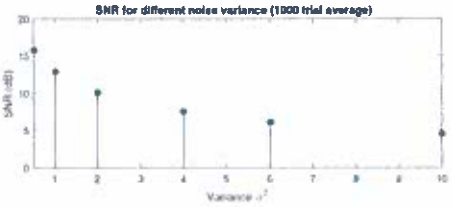
Question 2

Observation of **figure 4.1.4** shows a positive linear relationship between the increase in sum of squared error and the increase in additive noise $\eta[n]$ variance and an expected decrease in SNR for increased additive noise $\eta[n]$.

Correct SNR calculation



(a) Additive noise variance against SSE for $\mathbf{w}_{opt}$ compared with known b



(b) Additive noise variance against SNR ratio of measured output.

Figure 4.1.4: Additive noise ($\eta[n]$) variance against sum of squared error and signal to noise ratio.

Question 3

The Wiener solution: $\mathbf{w}_{opt} = \mathbf{R}_{xx}^{-1}\mathbf{p}_{zx}$, requires an autocorrelation, a cross correlation, an inverse matrix operator, and matrix multiplication to compute.

comprehensive estimation of Complexity

Given filter coefficients of number $p+1$,
the autocorrelation and cross correlation share the same complexity:

$$Additions: \quad (p+1)\cdot N, \quad Multiplications: \quad (p+1)\cdot N$$

The inverse matrix operation has a complexity of the order $\mathcal{O}((p+1)^3) \approx \mathcal{O}(p^3)$, although this can be reduced to $\mathcal{O}(p^{2.376})$ using the Coppersmith–Winograd algorithm.[4]

The matrix multiplication:

$$Additions: \quad (p+1)^2 \approx (p^2), \quad Multiplications: \quad (p+1)^2 \approx (p^2), \quad Complexity = \mathcal{O}(p^2)$$

The matrix inversion is the highest order complexity, and therefore dominates. The complexity of the Wiener filter solution is of the order $O(p^3)$. This becomes very problematic if the autocorrelation matrix $\mathbf{R}_{xx}$ is large, so we must consider other iterative solutions such as the least mean square algorithm.

4.2 The least mean square (LMS) algorithm

The predicted filter weights now have a time varying form, so that they can be iterated towards a solution. By taking our earlier derivative $\frac{\partial J}{\partial w_k}$ from section 4.1 and making it time varying, we have:

$$\nabla_{w_k} J(n) = -r_{dx}(k) + \sum_{j=1}^{p} w_j r_{xx}(j,k)$$

By considering the method of steep descent towards the optimal solution, we make adjustments to the filter weights in the direction of the negative of the gradient:

$$\Delta w_k(n) = -\mu \nabla_{w_k} J(n)$$

Combination of these two equations yields:

$$w_k(n+1) = w_k(n) + \mu\left[r_{dx}(k) - \sum_{j=1}^{p} w_j r_{xx}(j,k)\right] = w_k(n) + \mu\left[d(n) - \sum_{j=1}^{p} w_j x_j\right]x_k(n)$$

$$w_k(n+1) = w_k(n) + \mu\Big[d(n) - y(n)\Big]x_k(n) = w_k(n) + \mu\Big[e(n)\Big]x_k(n)$$

Finally, in vector form we have:

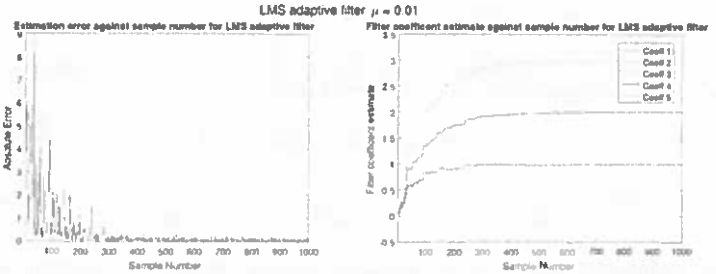
$$\mathbf{w}(n+1) = \mathbf{w}(n) + \mu e(n)\mathbf{x}(n)$$

Figure 4.2.1: Least mean squares adaptive filter algorithm in vector form.

Question 1

Observation of **figure 4.2.2** shows that the estimation error decreases as the filter coefficients tend towards their expected values of $\mathbf{b} = [1, 2, 3, 2, 1]$. In this particular case, the function `lms` uses a gain μ of 0.01. This gives the prediction a slow rise time but little overshoot.

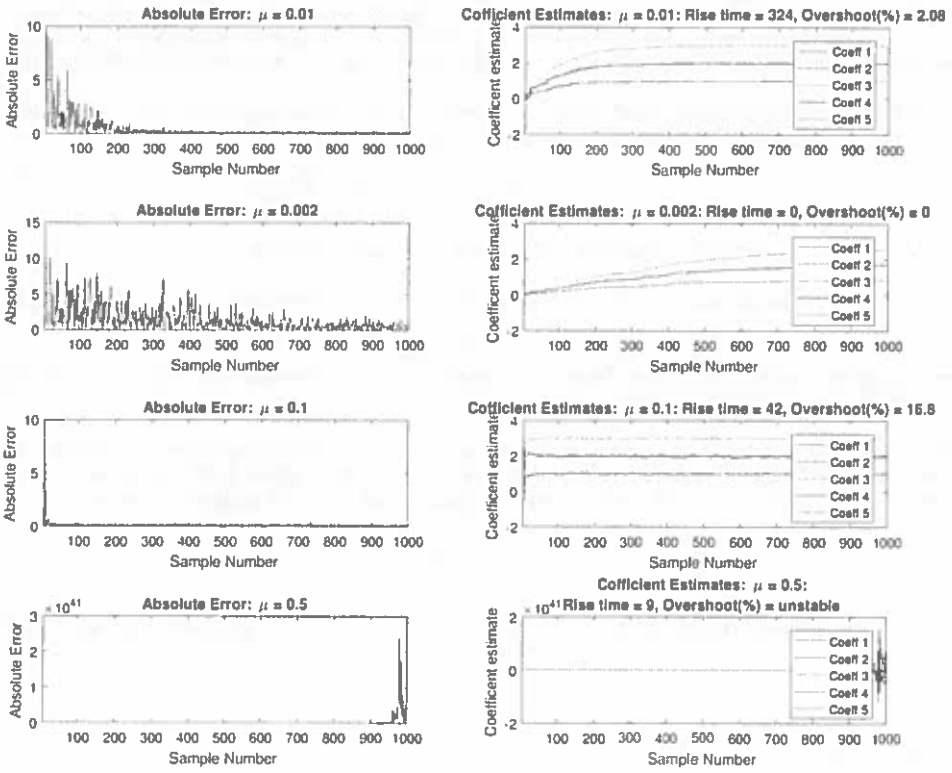
[4] "On the Complexity of Matrix Multiplication": Stothers 2010



Correct plots but title, legend and axis are too small

Figure 4.2.2: Least mean squares adaptive filter, used to predict filter coefficients using signals y and z from part 4.1.

Question 2



good explanation of tradeoff between convergence speed and steady state error.

Figure 4.2.3: Least mean squares adaptive filter with varying adaptation gain/step size μ, used to predict filter coefficients using signals y and z from part 4.1.

In this case the rise time as the longest time a single coefficient takes to reach 95% of its final value, in terms of sample number. As previously mentioned, the overshoot is defined as the maximum deviation of a filter coefficient prediction above its true value in percentage terms. Observation of **figure 4.2.3** shows that a gain of $\mu = 0.01$ converges slowly (rise time = 313 samples) and has minimal overshoot (2.08%). A gain of $\mu = 0.1$ converges much faster (rise time = 42 samples), and has significant overshoot (16.8%). On the two extreme ends, a gain of $\mu = 0.002$ is so slow to adapt that it doesn't converge within 1000 samples (rise time specified as 0), and a gain of $\mu = 0.5$ makes the system unstable and the coefficients reach values of order 10^{41}.

Question 3

The computational complexity of the LMS algorithm can be split up into 2 main parts, calculating $\hat{y}$ (the output of our adaptive filter), and the change made to the filter coefficients $\mathbf{w}(n)$.
Computation of $\hat{y}$:

$$Additions = p, \quad Multiplications = (p+1)$$

Comprehensive estimation of complexity

Updating the filter coefficients:

$$Additions = (p+1), \quad Multiplications = (p+1)$$

We also have 1 extra addition to calculate the error, and 1 extra multiplication for the calculation of $\mu e(n)$. Therefore at each iteration the complexity is $2p+2$ additions, and $2p+3$ multiplications.

4.3 Gear shifting

Numerous gear shifting ideas are presented

Gear shifting involves changes to the adaptation gain μ dependent on the error before. Ideally, the algorithm will converge very fast, but have minimal steady state error after converging. In practice with a constant gain μ this is very difficult to achieve. We have seen that when μ is large, the LMS will converge fast but there will be large steady state error. The opposite occurs when μ is small, as there is less steady state error but the LMS will converge slowly. Therefore it is feasible that the gain can be high originally so that the LMS converges fast, and then once convergence occurs μ can be reduced to lessen the steady state error. This only applies when the data is stationary, as the adaptation gain will reduce to a point at which the adaptive filter will no longer adapt sufficiently to new stimuli.

The gear shifting algorithm implemented in this coursework includes a large initial value of gain, followed by a time decaying gain once convergence criterion has been met. These conditions are given by:

$$\mu = \begin{cases} input & \text{if } |e(n)| > \frac{|e(4+p)+e(3+p)+e(2+p)+e(1+p)|}{25} \\ \frac{30\,\mu}{n} & \text{if } |e(n)| < \frac{|e(4+p)+e(3+p)+e(2+p)+e(1+p)|}{25} \end{cases} \tag{20}$$

Where $N_w = (Order - 1)$, and the values of 25 and 30 were found experimentally.

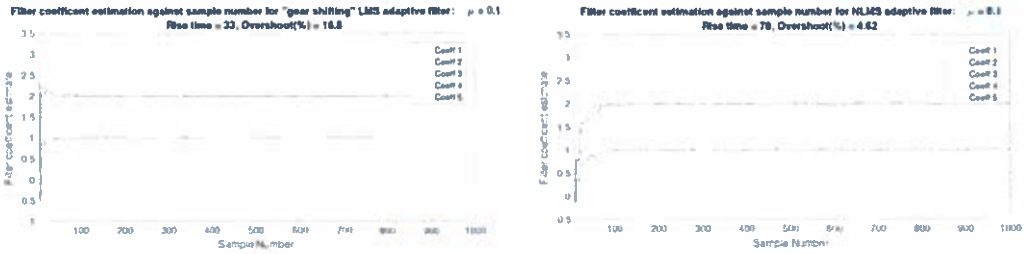
Limits can also be set on the gain μ to ensure stability. The condition for convergence in the mean is given by:

$$0 < \mu < \frac{2}{\lambda_{max}}$$

Where λ_{max} is the largest eigenvalue of the autocorrelation matrix $\mathbf{R}_{xx}$. Note that this does not ensure stability in the resulting coefficients, only stability in the mean.

A second method of adapting the gain is to normalize it with the power of the input data $\mathbf{x}$. This is useful as it prevents the correction from becoming large and unstable if the value of $\mathbf{x}$ increases significantly. This method is the normalized least mean square (NLMS) algorithm (which will be revisited in more detail in section 5), given by:

$$\mu(n) = \frac{\mu}{\|\mathbf{x}(n)\|_2^2}$$



(a) Gear shifting LMS algorithm

(b) Normalized LMS algorithm.

Figure 4.3.1: Comparison of LMS adaptive step size methods for filter system $\mathbf{b} = [1, 2, 3, 2, 1]$.

Figure 4.3.1(a) shows very fast convergence (rise time = 33) and very flat steady state, compared with the NLMS in **b** and the LMS algorithm previously. The NLMS does however show less immediate overshoot, as it approaches the true filter values with less speed.

4.4 Identification of AR processes

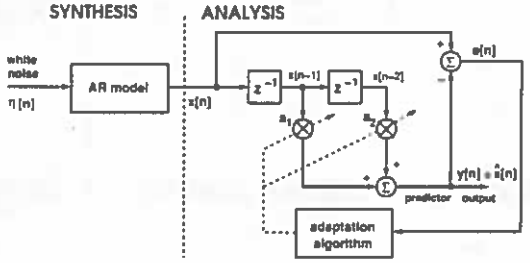


Figure 4.4.1: Adaptive estimation structure for AR(2) model prediction. [5]

Question 1

By implementing the structure in **figure 4.4.1**, an estimate of AR(2) process coefficients can be achieved. In this particular case the autoregressive process to be implemented is a = [1, 0.9, 0.2]. The expected output for a1 and a2 can be derived as follows:

$$X(z) = \frac{1}{1+0.9+0.2}\eta(z)$$
$$x[n]+0.9x[n-1]+0.2x[n-2] = \eta[n]$$
$$x[n] = \eta[n]-0.9x[n-1]-0.2x[n-2]$$

From the analysis structure in **figure 4.4.1** we have:

$$e[n] = x[n]-a_1x[n-1]-a_2x[n-2]$$
$$= \eta[n]-0.9x[n-1]-0.2x[n-2]-a_1x[n-1]-a_2x[n-2]$$

The LMS algorithm aims to eliminate error, thus setting $e[n] = 0$ (and removing $\eta[n]$ because $E\{\eta[n]\} = 0$) gives:

$$-0.9x[n-1]-0.2x[n-2]-a_1x[n-1]-a_2x[n-2] = 0$$
$$\therefore a_1 = -0.9, \quad a_2 = -0.2$$

Figure 4.4.2: Predicted values of a_1 and a_2 from correct implementation of AR(2) model analysis structure.

Correct plot of the evolution of the AR coefficients

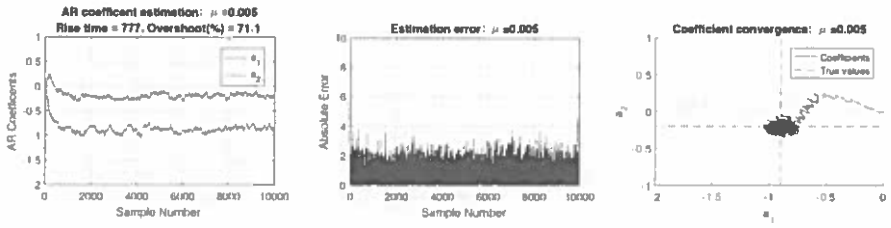


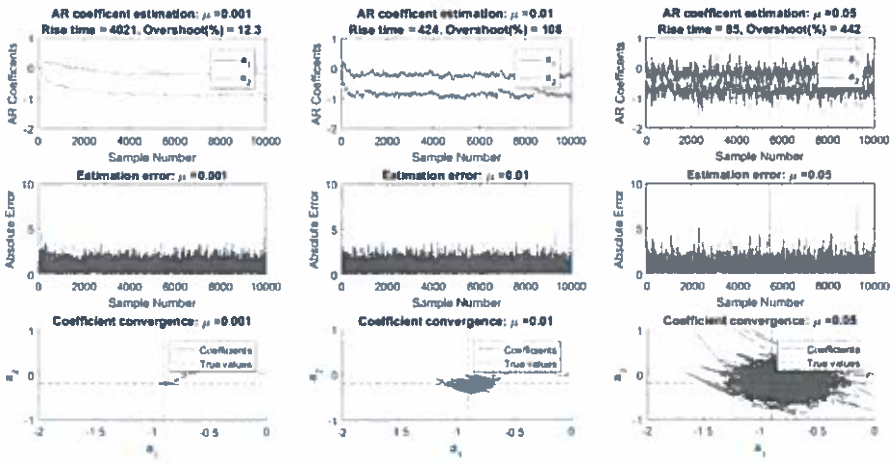
Figure 4.4.3: AR(2) (a = [1, 0.9, 0.2]) coefficient estimation using LMS adaptive filter structure.

Observation of **figure 4.4.3** shows that the coefficients a_1 and a_2 clearly tend towards their expected values of -0.9 and -0.2 respectively. The estimation error also represents the WGN $\eta[n]$ for most of the 10000 samples. This was also predicted above, as when $a_1 = -0.9$ and $a_2 = -0.2$, $e[n] = \eta[n]$.

Question 2

From **figure 4.4.4** we see that the lowest gain of $\mu = 0.001$ has the smallest and only reasonable overshoot of %12.3. However, this is accompanied by a very long rise time of 4021 samples. $\mu = 0.01$ converges much faster (rise time is 424 samples) but this is accompanied by a poor overshoot (108%). $\mu = 0.05$ has a very short rise time (82 samples), but it is heavily unstable and fluctuates massively, producing an overshoot of 442%. Interestingly, all of the error plots look relatively similar despite the stark differences in coefficient estimation. This once again is due to the noise term

[5] D.P.Mandic Advanced Signal Processing coursework 2017 page 16



Good explanation of tradeoff between convergence speed and steady state error.

Figure 4.4.4: AR(2) coefficient estimation using LMS adaptive filter structure, with varying adaptive gain (μ).

$\eta[n]$ dominating the error $e[n]$.

In the case of this system, we are responsible for feeding the noise into the unknown system, and therefore we can fully categorize it. If we can fully categorise the input noise $\eta[n]$ then we can remove it from our output. This is shown in figure 4.4.5 with an almost none existent overshoot of $1.94x10^{-}13$. This is an ideal scenario as it doesn't take into any measurement distortion or noise that may develop throughout the system in a real world situation. Nevertheless, it is important to see the difference that noise makes to the coefficient estimation and the corresponding error.

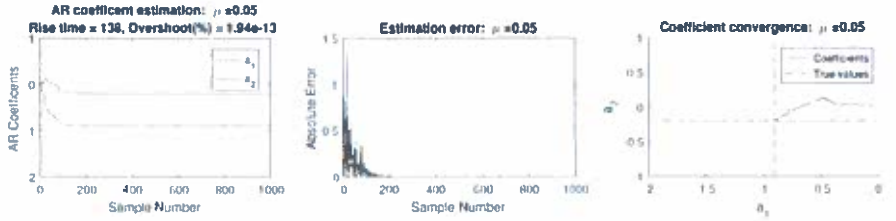


Figure 4.4.5: AR(2) coefficient estimation using LMS adaptive filter structure with input noise removed from the output. ($\mu = 0.05$).

4.5 Speech recognition



Correct plot of the time domain signals of the speech but title and axis are small

Figure 4.5.1: Personal audio recording of the letter X using the microphone on a 5th generation iPod Touch.

Observation of the 1000 sample section of 'X' in figure 4.5.1 shows the potential for an autoregressive speech model. The autocorrelation function appears to be a sum of decaying exponentials, which is characteristic of an AR model.

Question 1

Nice Plot of AR coefficients

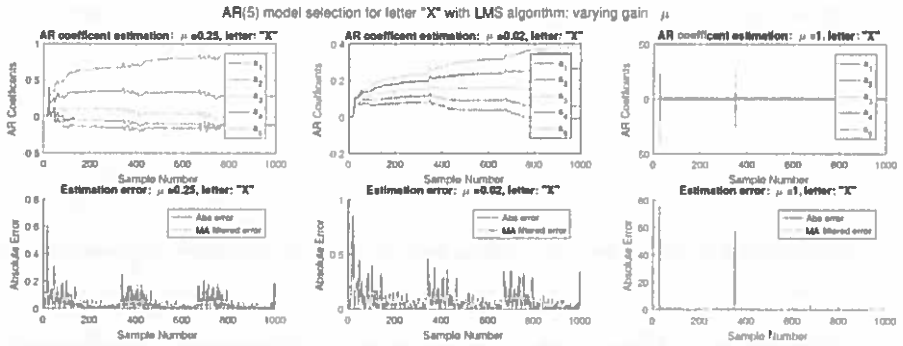


Figure 4.5.2: AR(5) coefficient estimation for audio recording of letter "X" with varied adaptive gain μ

Figure 4.5.2 suggests that the magnitude of the gain should be approximately 0.25. This produces the least error over 1000 samples, and appears to converge the fastest.

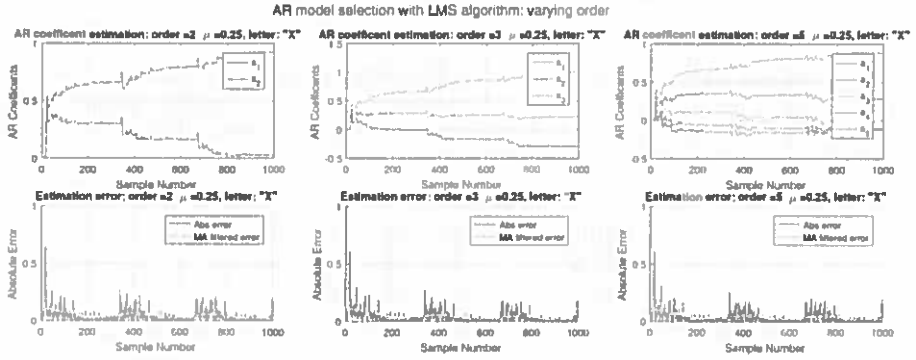


Figure 4.5.3: AR coefficient estimation for audio recording of letter "X" with varied AR model order

From **figure 4.5.3** the optimal order is unclear. The error decreases slightly with each increase in order, but this may be offset by the increase in computational complexity.

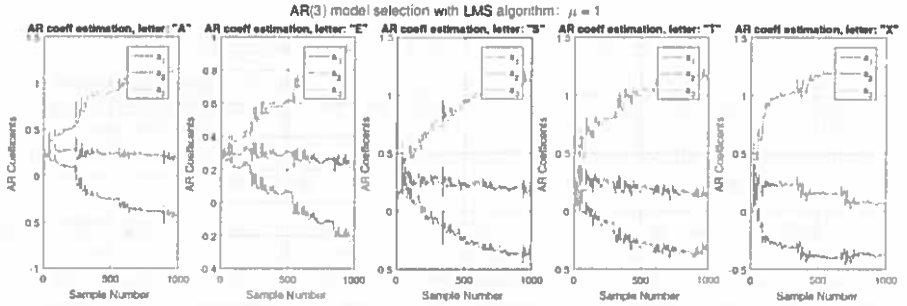


Figure 4.5.4: AR(3) coefficient estimation for audio recordings of all letters, adaptive gain $\mu = 1$.

Every letter other than E appears to be converging towards similar AR coefficients of $\mathbf{a} = [1, 0.4, -0.1, -1.2]$. It is expected that the coefficients are relatively similar, as all speech recordings are taken from the same voice within 5 minutes of each other. This allows the frequency range of the first and second formants, along with other important unique characteristics of the voice to remain relatively constant.

Question 2

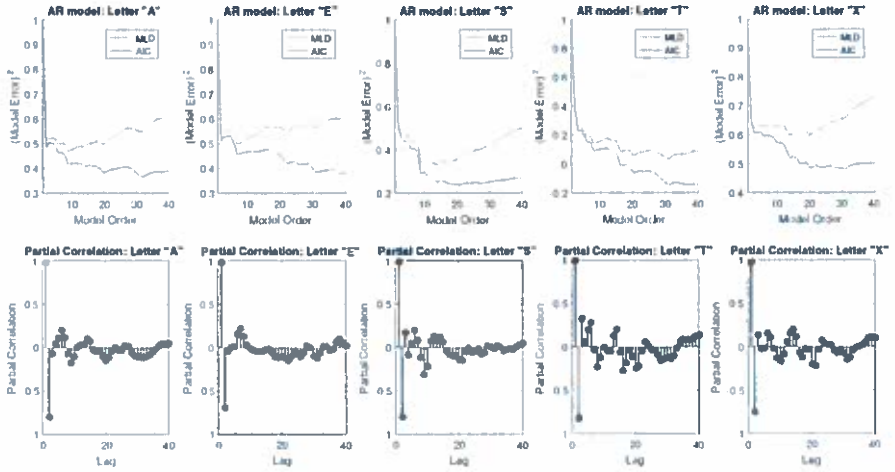


Figure 4.5.5: Minimum length description, Akaike information criterion, and partial correlation coefficients for all letters.

From observation of **figure 4.5.5** it is unclear what the optimal order should be. It is however clear that the optimal order is relatively high. We see this from the partial correlation now decaying quickly, and the MID/AIC having minimums at very large orders. From the partial correlation coefficients, an argument can be made for an order of approximately 22 for letters A,S,T and X, and approximately 9 for the letter E.

	Letter				
Method	"A"	"E"	"S"	"T"	"X"
MLD	9	7	14	31	17
AIC	32	38	21	40	31

Table 4.5.1: Optimal AR model order according to AIC and MLD calculations

Question 3

Prediction gain (a standard measure of predictor performance) is given by:

$$R_p = 10log_{10}(\frac{\sigma_x^2}{\sigma_e^2})$$

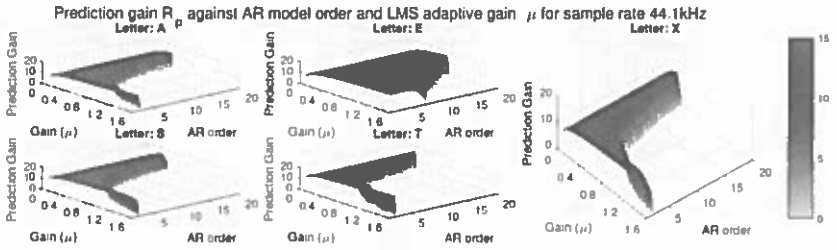
Figure 4.5.6: Formula for calculating prediction gain R_p.

It is clear from **figure 4.5.7** that with a lower sampling rate, the prediction gain is significantly lower across each letter. Therefore, we can say that the learning curves converge faster when the sampling rate of the sequence is higher. In terms of quasi-stationary vowel sounds, the sampling rate may decrease to a point where a lower number of samples are required to obtain a quasi-stationary sequence. This is because there is more time between each sample, and if too much time passes then the signal will change too much to be considered stationary.

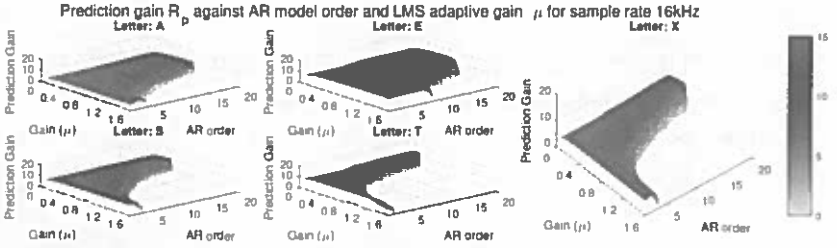
Furthermore, in **figure 4.5.7** we observe that the prediction gain is greater than 0 for more combinations of step size and order than the consonants. This suggests that the model order required is lower for the vowels, as most of the coefficients will therefore be close to zero at higher orders and the filter is less likely to be driven unstable by an increase in step size. This theory is further confirmed by our minimum length description calculations given in **table 4.5.1** which show that the order required for the vowels is lower than for the consonants.

In speech, consonants tend to exhibit rapid changes, whereas vowels exhibit slow changes[6]. The stationary approximation of a vowel is therefore more accurate than the than the stationary approximation for a consonant, and this in practice leads to a lower required AR model order.

[6]UCL: Introduction to Speech Science, Lecture 6, 2017



(a) Prediction gain: 44.1kHz



(b) Prediction gain: 16kHz

Figure 4.5.7: Prediction gain compared with AR model order and LMS adaptive gain, for each letter and different sampling frequencies.

4.6 Dealing with computational complexity: sign algorithms

Computation of least mean squares sign algorithms is far simpler than it is for standard least mean squares. This is very important in areas such as high speed communications, where it is desired to have minimal delay. The algorithms are given by:

$$\begin{aligned} signed-error: &\quad \mathbf{w}(n+1) = \mathbf{w}(n) + \mu sign(e[n])\mathbf{x}[n] \\ signed-regressor: &\quad \mathbf{w}(n+1) = \mathbf{w}(n) + \mu e[n] sign(\mathbf{x}[n]) \\ sign-sign: &\quad \mathbf{w}(n+1) = \mathbf{w}(n) + \mu sign(e[n]) sign(\mathbf{x}[n]) \end{aligned}$$

Figure 4.6.1: Signed least mean squares algorithms.

Observation of **figure 4.6.2(a)** in comparison with **figure 4.4.3**, shows that with the same gain of $\mu = 0.005$ the sign algorithms under perform against the standard LMS algorithm. Overshoot in general is higher with the signed algorithms, and the rise time is also far greater. This is to be expected however, as the benefit from signed algorithms is their lower complexity and therefore less computational cost. This lack of complexity against the standard LMS comes at the price of poorer accuracy.

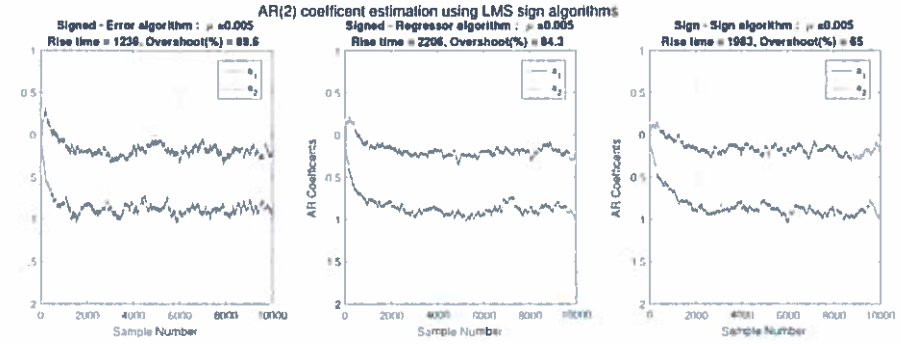
Figure 4.6.2(b) shows that in this particular case of AR(3) modelling of the letter "X", with $\mu = 0.5$, the LMS signed regressor algorithm out performs the other algorithms. Furthermore the Sign-Sign algorithm performs the poorest, having very large steady state error. The sign-sign algorithm is also the most computationally simple, so should be expected to have larger steady state error.

	Parameter		
LMS algorithm	**Overshoot (%)**	**Rise time (samples)**	**Mean Squared Error**
Signed - error	86.4	1280	1.028×10^{4}
Signed - regressor	72.0	1880	1.024×10^{4}
Sign - sign	76.7	2220	1.038×10^{4}

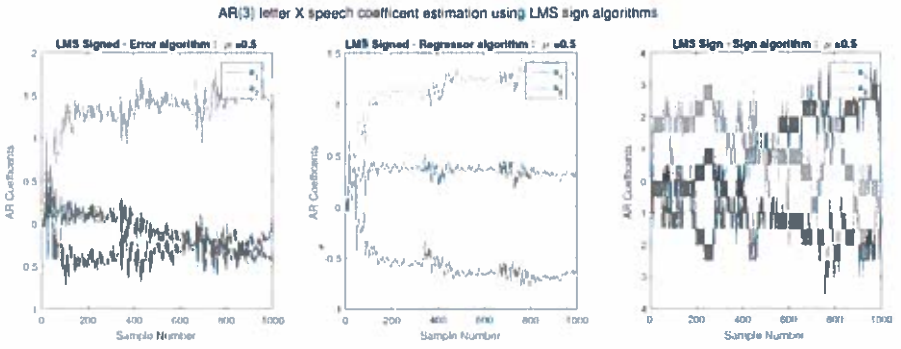
Table 4.6.1: 100 sample averaged performance parameters for signed LMS AR(2) prediction

Table 4.6.1 shows that the rise time, and mean squared error, increase heavily for the Sign-Sign algorithm compared with the signed-error and signed-regressor algorithms. This decrease in convergence rate and increase in error is met with no significant change in overshoot. As explained above, this is the price to pay for a reduction in computational complexity.

Correct plots of evolution of coefficients for 3 algorithms



(a) Part 4.4 AR(2) model (a = [1, 0.9, 0.2])



(b) Letter "X" part 4.5 speech file.

Figure 4.6.2: AR coefficient prediction for AR(2) and letter "X" using LMS signed algorithms.

5 A Real World Case Study: Vinyl Denoising

Dirt or a scratches on a vinyl record can corrupt the sound quality of the record as the record spins, creating a sharp sound referred to as a "tick" each time the damaged area passes the head that touches the record. In this section, we analyse a section of a vinyl recording of *Stairway to Heaven* by *Led Zeppelin*, which has been corrupted by these "ticks".

Question 1

Correct periodogram plot of the 2 channels

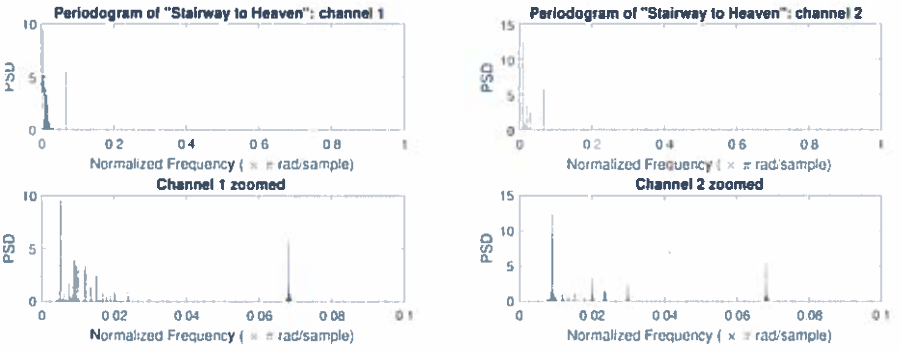


Figure 5.1: Periodograms of a corrupted vinyl of*Stairway to Heaven - Led Zeppelin*, channel 1 and channel 2.

From observation of **figure 5.1** it is clear that a spectral component likely corresponding to a tick is located at approximately $0.0068\pi rad/sample$, as this sharp spike is far from the rest of the spectrum and appears in both channels. Any other distortion is unclear from observation of the corrupted spectrogram alone.

Question 2

Correct Tick identification explanation

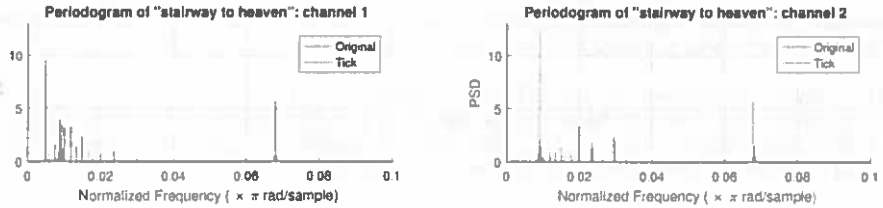


Figure 5.2: Periodograms of original *Stairway to Heaven - Led Zeppelin*, channel 1 and channel 2, with ticks overlaid in red.

Given the original uncorrupted recording, we can characterise the spectrum of the ticks by finding the difference between the corrupted recording and the uncorrupted recording. **Figure 5.2** shows the difference periodogram overlaid on a plot of the original uncorrupted recording. It is clear that on channel one there is one spectral component corresponding to the ticks (at $0.06802\pi rads/sample$) and on channel two there are two spectral components corresponding to the ticks (at $0.06802\pi rads/sample$, and $0.00907\pi rads/sample$).

Question 3

To filter out the spectral components of the ticks from the vinyl recording, it could be argued that a low pass filter would suffice to filter out the tick at $0.06802\pi rads/sample$, and a high pass filter would suffice to filter out the tick at $0.00907\pi rads/sample$. However, this disregards any spectral information from the music that may fall above all below these points on the periodgram. Removal of these points could distort the music, and therefore it is best to proceed with notch/bandstop filters which filter out a specific frequency component and let all other frequencies pass.
The second order IIR system function is given by:

$$H(z) = \frac{(z - e^{jw_0})(z - e^{-jw_0})}{(z - re^{jw_0})(z - re^{-jw_0})}$$

Using Euler's identity of $e^{j\theta} = cos\theta + jsin\theta$, the transfer function becomes:

$$H(z) = \frac{1 - 2cosw_0 z^{-1} + z^{-2}}{1 - 2rcosw_0 z^{-1} + r^2 z^{-2}}$$

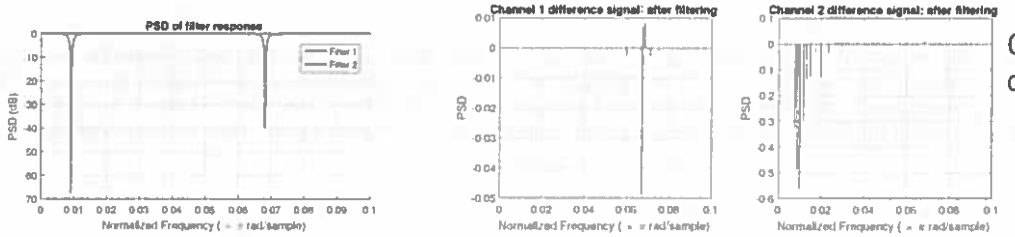
where, w_0 represents the notch frequency of the filter.

Reasonable justification of the filter parameters

Another important consideration is the bandwidth of the filter. An increase in bandwidth allows for a larger dB drop off at the notch, and it can be increased when filtering out the tick at $0.06802\pi rads/sample$, because there isn't a large density of spectral components surrounding it. However, for the tick at $0.00907\pi rads/sample$ which is in a higher density area, a smaller bandwidth must be considered to minimise the loss of spectral information surrounding the tick.

Figure 5.3(a) shows the magnitude response of filters 1 and 2 for each tick frequency, with the same bandwidth. Filter 2 experiences far greater dB drop at the notch frequency. This is because filter 2 has a pair of zeros that are far closer together on the unit circle, and therefore both zeros influence eachother more, causing a greater dB drop than filter 1 which is at a higher frequency. It is clear from **(b)** that filter 1 does not have the require dB drop to fully remove the tick, and therefore the bandwidth must be increased.

Correct application of notch filter



(a) Magnitude response of filters with the same bandwidth

(b) PSD difference signals for both channels (Original - filtered), for filters with the same bandwidth.

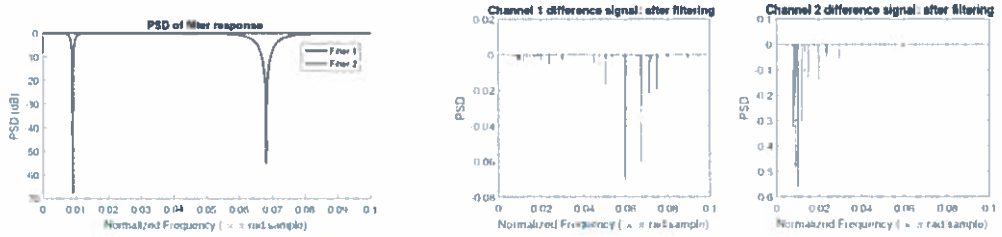
Figure 5.3: Magnitude response of filters with the same bandwidth, and accompanying difference signal.

Using the MATLAB function `iirnotch` with notch frequencies corresponding to the ticks, and increased bandwidth for filter 1, filter coefficients are found as:

$$Filter1 = \frac{0.9877 - 1.9304z^{-1} + 0.9877z^{2}}{1 - 1.95031z^{-1} + 0.9753z^{-2}} \quad Filter2 = \frac{0.9978 - 1.9949z^{-1} + 0.9978z^{2}}{1 - 1.9949z^{-1} + 0.9957z^{-2}}$$

These filters were also cascaded to increase the dB drop further. Channel 1 was filtered with a cascade of 3x filter 1, and channel 2 was filtered with a cascade of 3x filter 1 and 5x filter 2.

Observation of **figure 5.4(a)** shows that through increasing the bandwidth of filter 1, the dB drop has significantly improved. This is evident in **(b)** which shows the total removal of the ticks from the spectra. That being said, there is a distinct loss of spectral information that doesn't include the ticks. Qualitatively this has not effected the quality of the song, but it has increased the relative error that will be shown in *question 4*.



(a) Magnitude response of filters, with increased bandwidth for filter 1

(b) PSD difference signals for both channels (Original - filtered), with increased filter 1 bandwidth.

Figure 5.4: Magnitude response of filters with increased filter 1 bandwidth, and accompanying difference signal.

Question 4

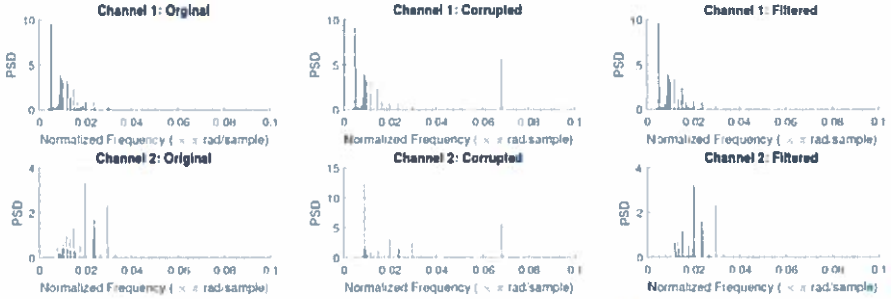


Figure 5.5: Periodograms of original, corrupted and filtered signals from channel 1 and 2 (blue, red and green respectively).

Figure 5.5 shows that the filtered signal (green) has that the ticks removed successfully, and it therefore resembles the original signal (blue) almost perfectly, bar a slight reduction in spectral magnitude and a slight loss of spectral information on channel 2 and the point of the $0.00907\pi rads/sample$ tick. This is quantified in **table 5.1** which shows a great reduction in the relative error of filtered signals compared with corrupted signals. Relative error is given by:

$$\frac{\|P_c - \hat{P}_c\|}{\|P_c\|}$$

where P_c is the original (non corrupted) periodogram, and $\hat{P}_c$ is the filtered or corrupted periodogram.

Correct Relative Error Computation

	Relative Error (unitless)	
Signal	**Channel 1**	**Channel 2**
Corrupted	0.2275	1.3296
Filtered	0.0115	0.2084

Table 5.1: Relative Error R_E for corrupted and filtered signals, for both channels

Question 5

Rather than manually selecting filters and their coefficients, one can use a supervised adaptive denoising algorithm which uses the original clean recorded signal as a comparison to adapt the filter coefficients to filter out the ticks from the corrupted recording. The simplest is a standard LMS adaptive algorithm which was derived in subsection 4.2. One can compare the prediction gain R_p for varying stepsize μ and filter order (as done in subsection 4.5), to arrive at an optimal configuration for this adaptive filter.

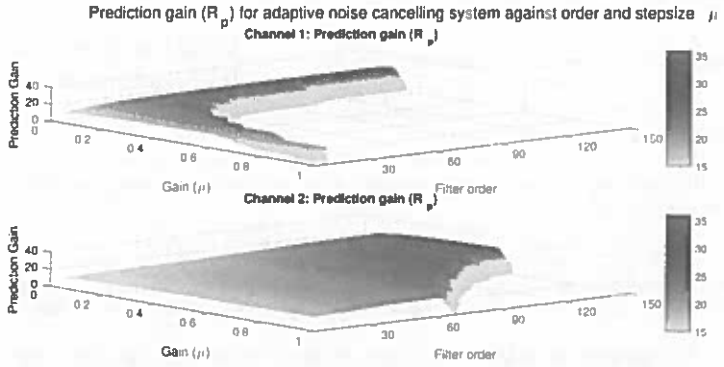


Figure 5.6: LMS prediction gain R_p against filter order and step size μ for both channel 1 and channel 2.

The prediction gain calculations shown in **figure 5.6** reveal that in the range of gain and order that has been examined, the optimal configurations for channel 1 and channel 2 are order = 95, μ = 0.2, and order = 150, μ = 0.5 respectively. In the case of the LMS, the maximum prediction gain for both channels is similar (channel 2 = 27.5, channel 1 = 26.5), but channel 2 remains stable for far more coefficient and order combinations.

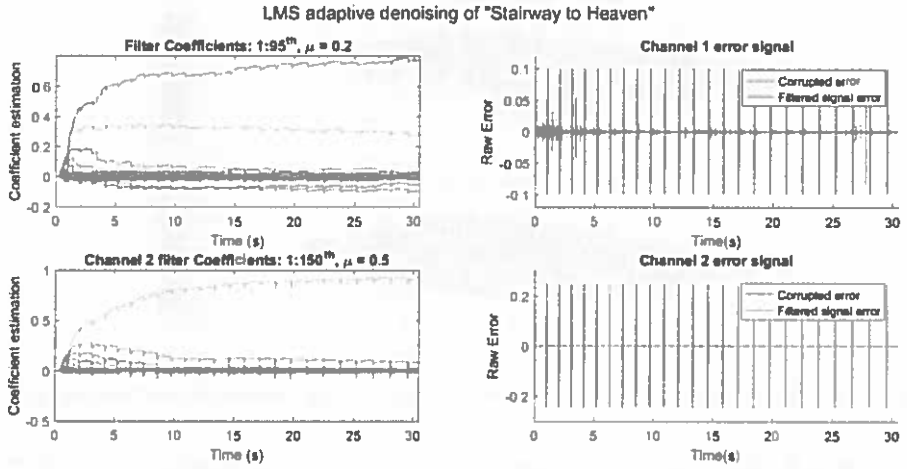


Figure 5.7: LMS adaptive coefficients and error against time for channel 1 and channel 2.

Observation of **figure 5.7** shows that for both channels, the majority of coefficients are small and cluster around zero. This is to be expected, as 150 and even 95 coefficients are very large orders, and the filter is unlikely to fully require this many degrees of freedom. Despite this, it does improve the accuracy of the estimate and seeing as the sample size is so large (1345050 samples), we can proceed with these orders. It is also observed that channel 1 has a very poor error reduction compared with channel 2, as channel 1 has error which is heavily influenced by each tick. Over all we still witness a large reduction in the error between the corrupted and filtered signal, and this is further reinforced in **figure 5.8** which shows a major removal in the spectral components corresponding to the ticks.
In the case of this noise canceling system it may be more optimal to use the normalised LMS algorithm, as the input power will increase at each tick. It makes sense therefore to normalise our correction by input signal power, to prevent the power spikes from negatively impacting the convergence of the filter coefficients.

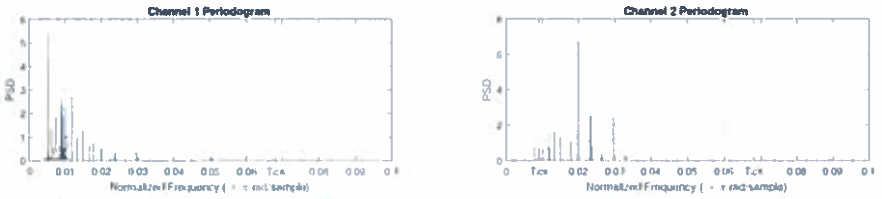


Figure 5.8: Periodogram of LMS adaptive noise filtered signals for each of channel 1 and channel 2, taken from the last half of the samples which assumes filter convergence.

Good Comparison of the NLMS vs LMS

The normalised LMS algorithm introduces a normalised step size of:

$$\mu(n) = \frac{\mu}{\|\mathbf{x}(n)\|^2}$$

where $\mathbf{x}(n)$ is the input signal to the filter, and $0 < \mu < 2$.

This ensures stability of the mean-square, and not just the mean as described earlier in subsection 4.3. The full NLMS algorithm is therefore given by:

$$\hat{y}[n] = \mathbf{w}_n^T \mathbf{x}_n, \qquad \mathbf{w}_{n+1} = \mathbf{w}_n + \mu(y[n] - \hat{y}[n]) \frac{\mathbf{x}_n}{\|\mathbf{x}(n)\|_2^2}$$

where in this case of adaptive noise canceling, $\mathbf{x}_n$ is the corrupted sample vector, $y[n]$ is the original non corrupted teaching signal, $\hat{y}[n]$ is the NLMS estimation, and $\mathbf{w}_n$ is the adaptive filter coefficients vector at time n.

A practical adaptation can also made to the NLMS in this form. An issue can occasionally occur when $\|\mathbf{x}(n)\|$ tends to zero, which would cause the adaptation to rise to infinity. Therefore a small constant ϵ is added to prevent this, and the implementation becomes:

$$\mathbf{w}_{n+1} = \mathbf{w}_n + \mu(y[n] - \hat{y}[n]) \frac{\mathbf{x}_n}{\epsilon + \|\mathbf{x}(n)\|_2^2}$$

We can perform the same R_p calculations as we did for the LMS algorithm to arrive at an optimal gain and order for the NLMS algorithm.

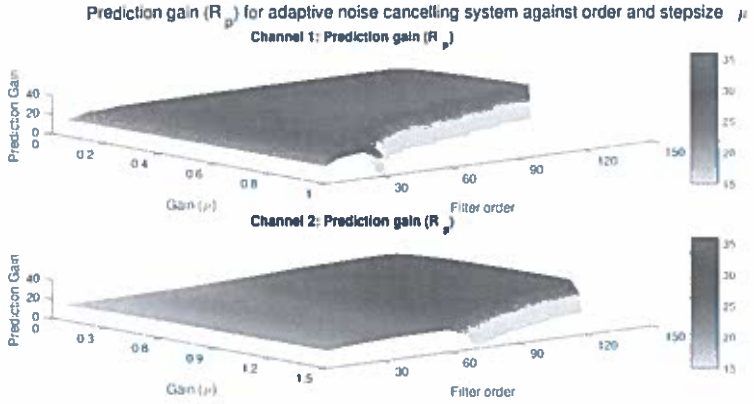


Figure 5.9: NLMS prediction gain R_p against filter order and step size μ for both channel 1 and channel 2.

Observation of **figure 5.9** shows that the highest prediction gain occurs at the highest order measured of 150, in both channels. In channel 1 the corresponding step size is $\mu = 0.55$, and in channel 2 the corresponding step size is $\mu = 1.05$. Channel 2 has a lower prediction gain in general, and this makes logical sense because the adaptive filter has to filter out 2 spectral tick components in channel 2, compared with 1 component in channel 1. The prediction gain for the NLMS is higher across the board when compared to the LMS algorithm.
Observation of **figure 5.10** shows that the majority of the 150 filter coefficients (in both cases) are very small and barely contribute to the filter. However, the negative cost of increased computational complexity for a slight benefit of increased accuracy does not have an impact in this case, as this is post processing. In post processing we can usually disregard otherwise bad increases in computational complexity as there are no strict time constraints. The error plot shows a large reduction in error between the corrupted signal and the filtered signal, and the error plot of channel 1 for the NLMS is a vast improvement compared with LMS channel 1 error. This is further reinforced with **figure 5.11** which shows removal of the spectral components associated with the ticks.
Quantitative comparisons of the LMS and NLMS performance can be made with relative error (explained in *question 4*) shown in **table 5.2**. It is clear that in both channels, the LMS is outperformed by the NLMS which has the smaller relative error. Qualitatively, through listening to the filter output, the improved performance of NLMS is also clear.

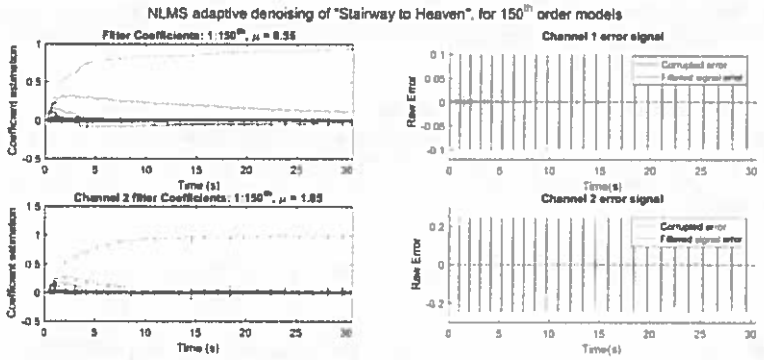


Figure 5.10: NLMS adaptive coefficients and error against time for channel 1 and channel 2.

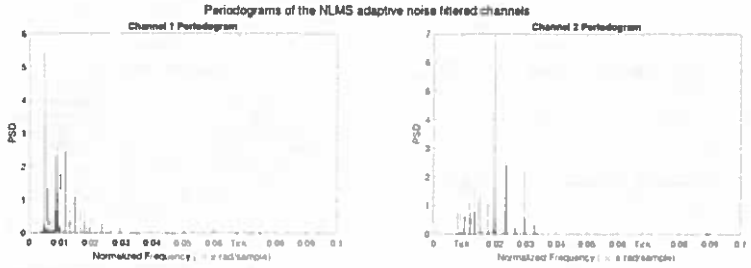


Figure 5.11: Periodogram of NLMS adaptive noise filtered signals for each of channel 1 and channel 2, taken from the last half of the samples which assumes filter convergence.

Furthermore, if we compare the relative error of the adaptive filtering to the relative error for the digital filters in **table 5.1**, we observe that it is far lower. Despite this, when listening to the filtered results using the MATLAB function **sound**, the ticks are still notably audible in the signals that have been subjected to adaptive denoising. An explanation for this is that the even though the digital filters remove a greater proportion of the tick from the spectrum, they also remove other spectral information which increases the relative error. This is an example of why qualitative and quantitative analysis must be used in conjunction for sound processing.

	Relative Error (unitless)	
Algorithm	Channel 1	Channel 2
LMS	0.0062	0.0097
NLMS	0.0013	0.0069

Table 5.2: Relative Error R_E for LMS and NLMS filtered signals, for both channels

The decrease in error of the NLMS comes with cost of higher computational complexity.
If the added term is calculated recursively as follows:

$$\|\mathbf{x}(n+1)\|^2 = \|\mathbf{x}(n)\|^2 + |x(n_1)|^2 - |x(n-p)|^2$$

Then the extra computation between LMS and NLMS will be 2 additions and 2 squaring operations. If we refer back to subsection **4.2**, then the complexity becomes $2p+5$ multiplications and $2p+4$ additions (with each iteration) for a filter that has $p+1$ coefficients.

Question 6

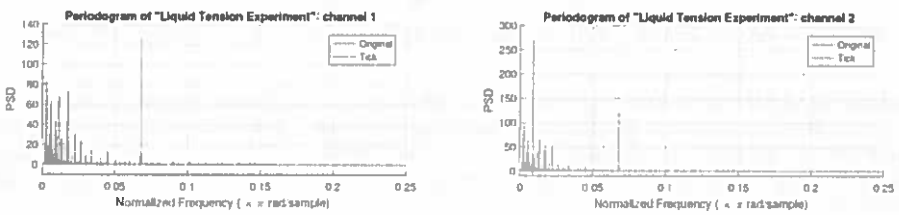


Figure 5.12: Periodograms of original *Liquid Tension Experiment*, channel 1 and channel 2, with ticks overlaid in red.

Figure 5.12 shows that the ticks have the same spectral position as they did in our previous analysis. However, it is clear that the song by *Liquid Tension Experiment* is far richer in frequency content, which may make it more difficult to remove the ticks with adaptive filtering.

Previous analysis showed that the NLMS outperformed the LMS algorithm, and therefore the adaptive denoising of *Liquid Tension Experiment* is also carried out with the normalised least mean squares method. To ensure that the filter doesn't become unstable and that it performs optimally, the order and step size μ are optimised again for this particular song. The optimisation yields a step size of $\mu = 0.2$ and an order of 150 for channel 1, and a step size $\mu = 0.15$ and an order of 150 for channel 2.

Correct and nice plots but legend and title are small

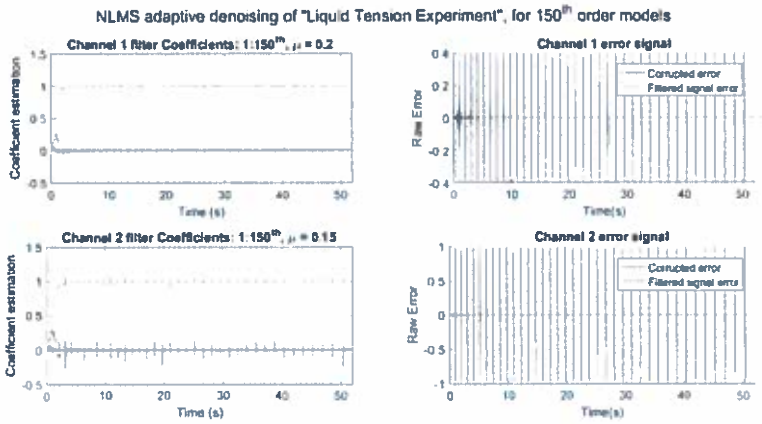


Figure 5.13: NLMS adaptive coefficients and error against time for channel 1 and channel 2, for *Liquid Tension Experiment*.

Figure 5.13 shows that the coefficients converge quickly for *Liquid Tension Experiment* and as was the case with the *Led Zeppelin* most of the coefficients are effectively zero at convergence. The plot of raw error shows a large reduction in error between the corrupted signal and the filtered signal, for both channel 1 and channel 2. This is reinforced by **figure 5.14** which appears to show the removal of the spectral components of the ticks.

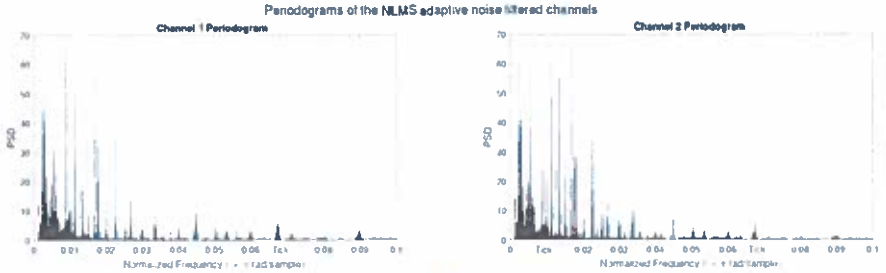


Figure 5.14: Periodogram of NLMS adaptive noise filtered signals for each of channel 1 and channel 2, taken from the last half of the samples which assumes filter convergence, for *Liquid Tension Experiment*.

	Relative Error (unitless)	
Algorithm	**Channel 1**	**Channel 2**
NLMS	0.0036	0.0198

Table 5.3: Relative Error R_E for NLMS filtered signals, both channels, for *Liquid Tension Experiment*

Table 5.3 shows an higher relative error from the adaptive filtering of *Liquid Tension Experiment* when compared with **table 5.2** for *Led Zeppelin*. Despite this increase in relative error, the audio playback of *LTE* yields an almost perfect response in which the ticks are not audible and the sound quality is high, which was not the case with *Led Zeppelin*. The reason for this is the high density of spectral information for *LTE*, which makes the removal of extra information along with the ticks almost unavoidable.